

Algoritmi per l'intelligenza artificiale

Simone Colli
`simone.colli@studenti.unipr.it`

Appunti del corso tenuto dal **Prof. Vincenzo Bonnici**

Università degli Studi di Parma
Anno Accademico 2025/2026

Indice

1	Introduzione all'apprendimento automatico	4
1.1	Apprendimento automatico supervisionato	5
1.2	Apprendimento automatico semi-supervisionato	8
1.3	Apprendimento automatico non supervisionato	10
1.4	Apprendimento per rinforzo	13
2	Background	14
2.1	Distribuzioni e teoria dell'informazione	15
2.1.1	Basi della teoria dell'informazione	15
2.1.2	Distribuzioni	15
2.1.3	Indici statistici delle distribuzioni	15
2.1.4	Confronto tra distribuzioni	15
2.1.5	Concetti applicati alla biologia	15
2.2	Casualità	15
2.2.1	Probabilità	15
2.2.2	Generatori	15
2.2.3	Istogrammi di frequenza	15
2.2.4	Test di casualità	15
2.2.5	Concetti applicati alla biologia	15
3	Classificazione	16
3.1	Introduzione alla classificazione	16
3.2	Costruire un classificatore	19
3.2.1	Proprietà di un classificatore	19
3.2.2	Il problema dell'overfitting	20
3.2.3	Validazione	20
3.3	Gestione delle feature e del rumore	21
3.3.1	Selezione delle feature	21
3.3.2	Rumore e outlier	21
3.4	Valutazione degli errori	22
3.5	Fasi di un sistema di classificazione	23
3.6	Tecniche di validazione per la classificazione	24
3.7	Modello di validazione base: Training e Test Set	25
3.8	Metriche di valutazione	26
3.9	Accuracy	29

3.10	Altri indici e matrice di confusione	30
3.11	Area Under the Curve (AUC) e curva ROC	31
3.12	Classificazione multi-classe	32
3.13	Cross-validation	35
4	Algoritmi di classificazione	36
4.1	Tipologie di modelli di classificazione	36
4.2	Approcci lazy	38
4.3	Approcci eager	40
5	Clustering	51
5.1	Introduzione al clustering	51
5.2	Algoritmi di clustering	51
5.3	Valutazione del clustering	51
6	Grafi	52
6.1	Teoria dei grafi	52
6.2	Algoritmi di ricerca sui grafi	52
7	Algoritmi e metaeuristiche	53
7.1	Algoritmi di ricerca	53
7.2	Metaeuristiche	53
8	Teoria dei giochi	54
8.1	Introduzione	54
8.1.1	Classificazione dei giochi	54
8.2	Algoritmi e applicazioni	56
8.2.1	Algoritmo Minimax	56
8.2.2	Potatura alfa-beta	60
8.2.3	Pseudocodice potatura alfa-beta	61
8.2.4	Altre problematiche da affrontare	62
8.2.5	Problema dell'orizzonte	62
8.2.6	Eccessiva fiducia nell'euristica	63
8.2.7	Eventi stocastici e ExpectiMAX	63
8.2.8	Ricerca ExpectiMAX	64
8.2.9	Algoritmo Paranoid	66
8.3	Apprendimento per rinforzo	68
8.3.1	Introduzione	68
8.3.2	Model learning	68
8.3.3	Ricerca basata su simulazione	69

Disclaimer

Nota

I seguenti appunti sono stati generati a partire dal materiale didattico e dalle note ufficiali del corso “Algoritmi per l'intelligenza artificiale”. Il contenuto è stato riorganizzato, formattato e parzialmente rielaborato con l'ausilio di AI al fine di creare un documento coeso e ottimizzato per lo studio.

Si raccomanda di fare sempre riferimento al materiale originale fornito dal docente per garantire la massima accuratezza.

Capitolo 1

Introduzione all'apprendimento automatico

Nel campo dell'apprendimento automatico classico, le attività sono tradizionalmente suddivise in quattro rami principali:

- Apprendimento supervisionato (supervised).
- Apprendimento semi-supervisionato (semi-supervised).
- Apprendimento non supervisionato (unsupervised).
- Apprendimento per rinforzo (reinforcement learning).

La distinzione primaria tra queste metodologie di ML risiede nel livello di disponibilità dei “dati di verità di base” (ground truth).

La **ground truth** è definita come la conoscenza preliminare dell'output che il modello dovrebbe produrre per un dato input, basata sull'osservazione diretta in contrapposizione all'inferenza.

1.1 Apprendimento automatico supervisionato

L'apprendimento automatico supervisionato ha come obiettivo l'apprendimento di una funzione che, dato un **campione di dati** e i relativi output desiderati, riesca ad approssimare la funzione sottostante che mappa gli input agli output.

Questa metodologia è comunemente applicata in due principali contesti:

- **Classificazione:** quando si desidera mappare l'input a etichette di output discrete.
- **Regressione:** quando l'obiettivo è mappare l'input a un output continuo.

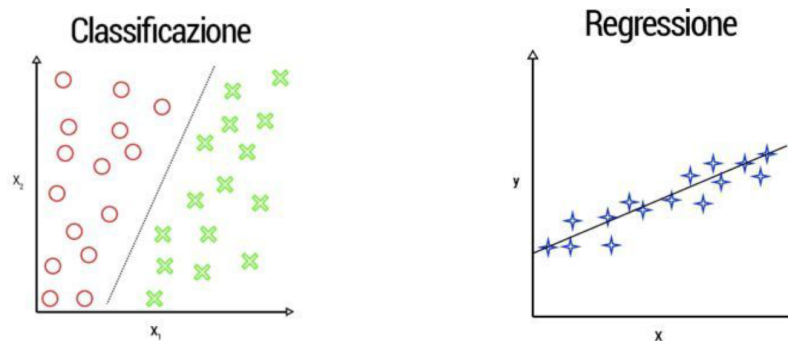


Figura 1.1: Esempi di apprendimento supervisionato: classificazione e regressione

In entrambi i casi, lo scopo è identificare relazioni o strutture specifiche nei dati di input che consentano di generare output corretti in modo efficace. È fondamentale notare che la correttezza dell'output è determinata interamente dai dati di addestramento, i quali costituiscono la “verità di base” che il modello apprende.

Nota 1.1.1: Importanza della qualità e della quantità dei dati

L'efficacia del modello può essere significativamente ridotta dalla presenza di etichette “rumorose” o “errate” all'interno dei dati stessi.

Oltre alle problematiche legate alla qualità dei dati, è importante considerare anche la quantità. In scenari in cui i dati di addestramento sono limitati o non uniformemente distribuiti, è cruciale optare per un modello a bassa complessità per evitare situazioni di overfitting.

Nota 1.1.2: Esempi di algoritmi supervisionati

Algoritmi notevoli nell'apprendimento supervisionato includono la regressione logistica, il classificatore bayesiano naif, le macchine a vettori di supporto, le reti neurali artificiali e le foreste casuali.

Nota 1.1.3: Complessità del modello

Il successo di un modello di ML dipende dalla sua capacità di generalizzazione. Questo concetto è strettamente connesso alla complessità del modello, che si riferisce alla complessità della funzione che si sta cercando di apprendere.

Overfitting

L'**overfitting** (sovradattamento) è una problematica molto comune che si verifica quando il modello apprende la funzione adattandosi troppo bene ai soli dati di addestramento, senza cogliere la tendenza o la struttura effettiva che guida l'output, e quindi non riesce a generalizzare a nuovi punti dati.

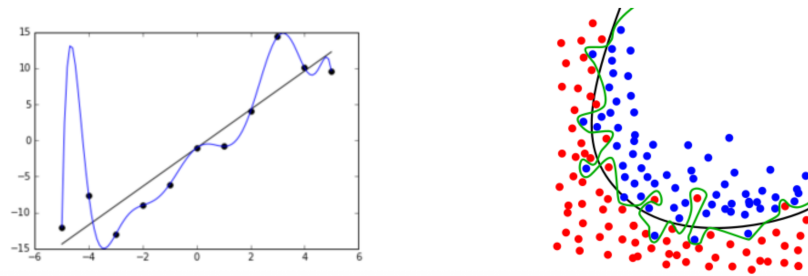


Figura 1.2: Esempi di overfitting

La gestione della generalizzazione è formalizzata tramite il compromesso **bias-varianza** (bias-variance tradeoff). Così facendo il modello presenterà un equilibrio tra:

- **Bias** (distorsione): l'errore sistematico dovuto a ipotesi errate nel processo di apprendimento.
- **Varianza**: la quantità in base alla quale l'errore può variare tra diversi set di dati.

La difficoltà si presenta nel creare un modello che cattura accuratamente le regolarità dei dati di addestramento e che sia in grado di generalizzare bene a dati non visti in precedenza.

In generale, un aumento del bias (e una conseguente riduzione della varianza) porta a modelli con livelli di prestazione più stabili e garantiti, un fattore che può essere cruciale in certe applicazioni.

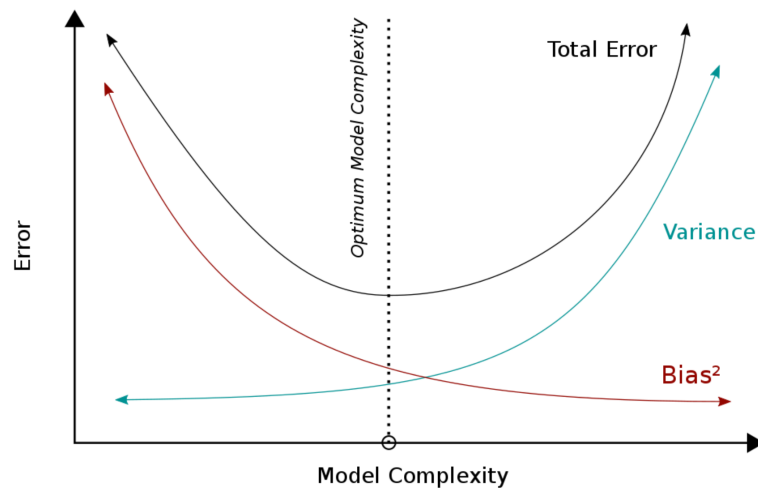


Figura 1.3: Compromesso bias-varianza

Per ottenere una buona generalizzazione, la varianza del modello deve essere attentamente bilanciata in base alla dimensione e alla complessità dei dati di addestramento.

Nota 1.1.4: Gestione dei dataset

Set di dati piccoli e semplici dovrebbero essere gestiti con modelli a bassa varianza, mentre set di dati grandi e complessi richiedono modelli con una varianza più elevata per poter catturare appieno la struttura sottostante dei dati.

1.2 Apprendimento automatico semi-supervisionato

L'apprendimento automatico semi-supervisionato (semi-supervised) mira a **etichettare un insieme di punti dati senza etichetta** sfruttando la conoscenza appresa da un piccolo numero di dati già etichettati.

Nota 1.2.1: Alcune metodologie comuni

Metodi comuni includono le **macchine vettoriali di supporto trasversali** e i **metodi basati su grafi**, come la propagazione delle etichette.

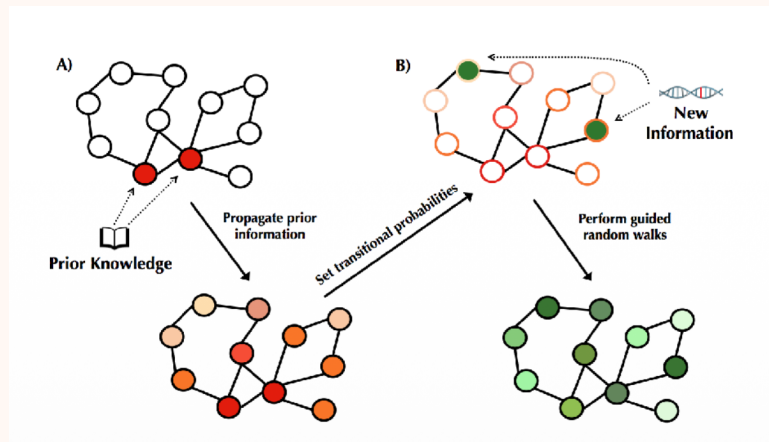


Figura 1.4: Esempio di apprendimento semi-supervisionato con propagazione delle etichette

Nota 1.2.2: Vantaggi dell'apprendimento semi-supervisionato

L'apprendimento semi-supervisionato può migliorare significativamente le prestazioni del modello rispetto all'apprendimento supervisionato quando si dispone di una quantità limitata di dati etichettati.

Tale limitazione può presentarsi in diversi scenari:

- Dati etichettati costosi da ottenere, ad esempio quando richiedono l'intervento umano.
- Dati etichettati difficili da ottenere, ad esempio quando richiedono competenze specialistiche.
- Dati etichettati soggetti a vincoli di privacy o etici.

Esempio 1.2.1: Rilevamento di messaggi inappropriati

Nel rilevamento di messaggi inappropriati in un social network, è impraticabile etichettare manualmente ogni messaggio. È possibile, invece, etichettare manualmente un piccolo sottoinsieme e usare tecniche semi-supervisionate per comprendere e classificare il resto dei contenuti.

Presupposti dell'apprendimento semi-supervisionato

Per poter giustificare l'utilizzo di pochi dati etichettati per trarre conclusioni su un grande insieme di dati non etichettati, i metodi semi-supervisionati si basano su alcuni presupposti fondamentali, quali:

- **Continuità:** Si assume che punti dati “vicini” tra loro abbiano maggiori probabilità di condividere la stessa etichetta.
- **Ipotesi del cluster:** Si presume che i dati formino naturalmente dei cluster discreti. Di conseguenza, punti nello stesso cluster hanno maggiori probabilità di condividere un'etichetta.
- **Presupposto molteplice (manifold):** Si ipotizza che i dati si trovino approssimativamente in uno spazio di dimensioni inferiori (un *manifold*) rispetto allo spazio di input originale.

L'ultimo presupposto è particolarmente rilevante in scenari quando un sistema con pochi parametri, non osservabile direttamente, produce output osservabili ad alta dimensione.

1.3 Apprendimento automatico non supervisionato

L'apprendimento automatico non supervisionato (unsupervised) opera **senza output etichettati**. L'obiettivo principale di questa tecnica è quello di **dedurre la struttura naturale** presente all'interno di un insieme di dati, cercando di trovare **pattern intrinseci** nei dati. Le attività più comuni in questo ambito includono:

- **Clustering** (raggruppamento).
- **Apprendimento della rappresentazione** (representation learning).
- **Stima della densità** (density estimation).

Nota 1.3.1: Valutazione delle prestazioni

Dato che non vengono fornite etichette, nella maggior parte dei metodi di apprendimento non supervisionato non esiste un modo specifico per confrontare le prestazioni del modello.

Clustering

Il clustering è una **tecnica esplorativa** che permette di aggregare dati in gruppi (detti *cluster*) senza avere una precedente conoscenza della loro appartenenza a tali gruppi. Si applica a dataset dove i dati al loro interno presentano elementi simili tra loro.

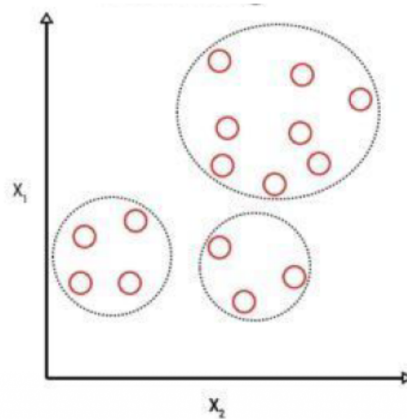


Figura 1.5: Esempio di clustering

All'interno di ogni singolo cluster si troveranno quindi dati che hanno molte **caratteristiche simili** tra loro. È un'ottima tecnica per trovare relazioni tra i dati.

Riduzione della dimensionalità

La riduzione della dimensionalità senza supervisione è un approccio molto usato nella **pre-elaborazione delle features**. L'obiettivo principale di questa tecnica è di **eliminare il “rumore”** dai dati. Un

altro utilizzo si trova nella **rappresentazione dei dati**: dati in uno spazio delle caratteristiche ad elevata dimensionalità possono essere proiettati su uno spazio 1D, 2D o 3D per l'analisi visiva.

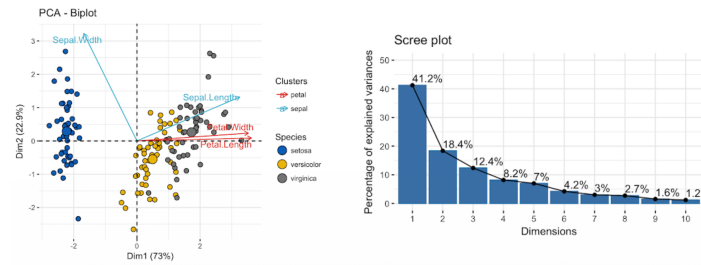


Figura 1.6: Esempio di PCA e scree plot

Nota 1.3.2: Compromesso tra riduzione della dimensionalità e prestazioni

Questa operazione può talvolta causare una minore prestazione predittiva. Ma può anche rendere lo spazio dimensionale più compatto, aiutando a **mantenere le informazioni più rilevanti**.

Analisi esplorativa

L'apprendimento non supervisionato è estremamente utile nell'**analisi esplorativa dei dati** (exploratory data analysis), poiché è in grado di **identificare automaticamente la struttura** nei dati.

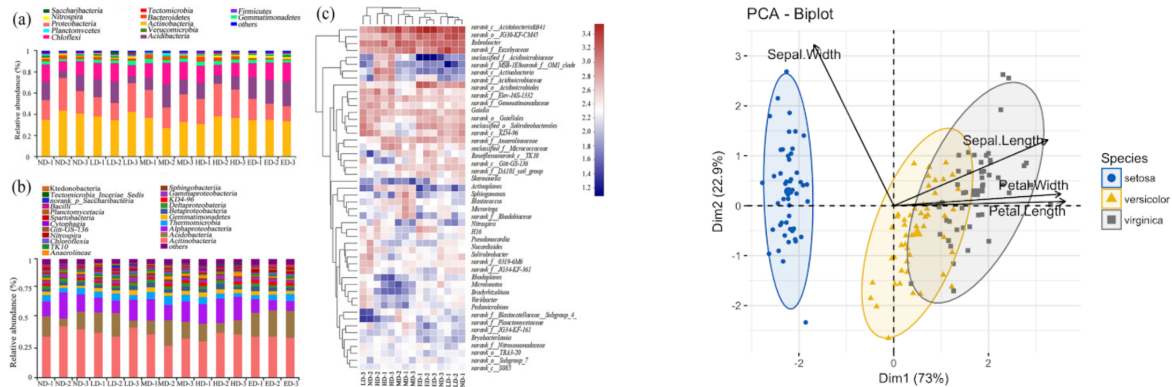


Figura 1.7: Esempio di analisi esplorativa con clustering gerarchico

Esempio 1.3.1: Segmentazione dei consumatori

Se un analista volesse segmentare i consumatori, i metodi di clustering sarebbero un ottimo punto di partenza per l'analisi.

Nota 1.3.3: Utilizzo in assenza di etichette

In situazioni dove è impraticabile o impossibile per un essere umano proporre tendenze nei dati, l'apprendimento non supervisionato può fornire **informazioni iniziali** che possono poi essere usate per testare o verificare singole ipotesi.

1.4 Apprendimento per rinforzo

L'apprendimento con rinforzo (reinforcement learning) ha l'obiettivo di realizzare **agenti autonomi**. Questi agenti devono essere in grado di scegliere azioni da compiere per conseguire determinati obiettivi. Questo avviene tramite l'interazione con l'ambiente in cui sono immersi, con lo scopo di massimizzare una nozione di **premio cumulativo**.

Capitolo 2

Background

2.1 Distribuzioni e teoria dell'informazione

2.1.1 Basi della teoria dell'informazione

2.1.2 Distribuzioni

2.1.3 Indici statistici delle distribuzioni

2.1.4 Confronto tra distribuzioni

2.1.5 Concetti applicati alla biologia

Paradosso evolutivo

DNA e trasmissione dell'informazione

Genomi come stringhe

Distribuzioni genomiche

2.2 Casualità

2.2.1 Probabilità

Probabilità discreta

Dadi ed eventi naturali multifattoriali

Monte Carlo per calcoli geometrici

2.2.2 Generatori

Altri generatori

2.2.3 Istogrammi di frequenza

2.2.4 Test di casualità

2.2.5 Concetti applicati alla biologia

Genomi random

Leggi genomiche informative

Stringhe random

Capitolo 3

Classificazione

3.1 Introduzione alla classificazione

La classificazione è un'attività dell'apprendimento supervisionato che consiste nell'assegnare una Definizione 3.1.3 (o Definizione 3.1.2) a un dato sulla base di sue caratteristiche osservabili (Definizione 3.1.1).

Definizione 3.1.1: Feature

Una **feature** (caratteristica) è un aspetto direttamente osservabile di un fenomeno per il quale si può registrare una misura, che sia quantitativa (numerica) o categoriale (come vero/falso, rosso/verde, ecc.).

Definizione 3.1.2: Classe

Una **classe** è un concetto astratto e generale che “spiega” le osservazioni. L'assegnazione a una classe costituisce una sintesi delle features (Definizione 3.1.1) osservate.

Definizione 3.1.3: Label

Una **label** (etichetta) è il nome specifico di una classe (Definizione 3.1.2).

Nota 3.1.1: Problematica della classificazione

Alcuni dati possono rendere più complesso l'assegnazione delle classi. Questa tipologia di dati è nota come **outlier statistici**.

Definizione 3.1.4: Collezione di dati

Una collezione di dati è un insieme di elementi P di M -uple del tipo:

$$m_i = (x_{1i}, \dots, x_{Mi}) \in D_1 \times \dots \times D_M$$

dove ogni x_{ji} rappresenta una feature (Definizione 3.1.1) ed appartiene ad un possibile dominio di valori D_j . Ogni feature (Definizione 3.1.1) x_{ji} appartiene a un dominio di valori D_j . I domini possono essere:

- **Numerici:** (es. $\mathbb{R}$ per valori continui, $\mathbb{Z}$ per discreti).
- **Categoriali:** Insiemi finiti di valori (es. {Rosso, Verde, Blu}).

Definizione 3.1.5: Partizionamento in classi

Data una collezione di dati (Definizione 3.1.4) P e un insieme finito di k **etichette** (Definizione 3.1.3) $L = \{A_1, \dots, A_k\}$.

Si dice che P è **partizionato in classi** se P è suddiviso in k sottoinsiemi $\{C_1, \dots, C_k\}$, tali che:

- $C_j \subseteq P, \forall j \in \{1, \dots, k\}$
- $C_i \cap C_j = \emptyset, \forall i \neq j$ (le classi sono mutuamente esclusive)
- $\bigcup_{j=1}^k C_j = P$ (l'unione delle classi copre l'intero dataset)

Ogni sottoinsieme C_j raggruppa tutti e soli gli elementi $m_i \in P$ che sono stati associati all'etichetta (classe) $A_j \in L$.

Definizione 3.1.6: Algoritmo di classificazione

Un **algoritmo di classificazione** è una funzione computabile $f : P \mapsto L$, tale che:

$$f(m \in P) = f(x_1, \dots, x_m) \in L$$

Tale funzione $f(m)$ assegna a ogni dato m un'etichetta (Definizione 3.1.3) A_i scelta tra quelle presenti in L cercando di stimare l'etichetta reale del stesso.

Lo schema di classificazione può produrre due tipi di risultati:

- **Successo** (hit) se l'etichetta stimata $f(m)$ coincide con l'etichetta reale del dato.
- **Fallimento** (miss) se l'etichetta stimata è errata.

Nota 3.1.2: Classificatori *error free*

È generalmente impossibile creare classificatori *error free*. Per questo motivo è fondamentale fornire stime sul tasso percentuale di hit/miss.

Il livello tollerabile di errore dipende dalla **criticità dell'applicazione**: per applicazioni industriali si può richiedere un tasso $< 5\%$, mentre per applicazioni mediche un tasso $> 0.5\%$ potrebbe essere già inaccettabile.

Esempio 3.1.1: Problema di classificazione: salmoni e branzino

Si consideri il problema di distinguere tra salmoni e branzini basandosi su alcune caratteristiche osservabili. Le feature (Definizione 3.1.1) utilizzate potrebbero essere la lunghezza, il peso in grammi e il colore dominante (un attributo qualitativo scelto da un insieme predefinito come {blu, grigio, verde}). I dati vengono tipicamente organizzati in una tabella, dove ogni riga corrisponde a un pesce e le colonne ne descrivono le feature (Definizione 3.1.1). L'obiettivo è costruire un classificatore che, per ogni nuovo pesce osservato, sia in grado di riempire la colonna "specie" con l'etichetta corretta ("salmone" o "branzino"). È importante notare che gli errori non hanno lo stesso costo: confondere un salmone (pesce pregiato) con un branzino (meno pregiato) è un errore più grave del contrario.

Esempio 3.1.2: Problema di classificazione: studenti e carriera

Si consideri il problema di predire il futuro successo economico degli studenti universitari. Le feature (Definizione 3.1.1) raccolte per ogni studente includono dati anagrafici, il censo familiare e i voti conseguiti durante la carriera universitaria. L'obiettivo è costruire un classificatore che predica in quale classe (Definizione 3.1.2) di reddito si troverà lo studente dieci anni dopo la laurea. Le label (Definizione 3.1.3) potrebbero essere {"reddito basso", "reddito medio", "reddito alto"}.

È importante notare che, a causa dell'elevato numero di fattori non misurabili che influenzano la vita di un individuo, una predizione del genere ha un valore limitato se applicata al singolo studente, che ha un'alta probabilità di essere classificato erroneamente.

Tuttavia, questo tipo di analisi è estremamente utile a livello statistico e aggregato, per comprendere le tendenze generali di un'intera popolazione studentesca e informare politiche educative o economiche.

3.2 Costruire un classificatore

Il processo di costruzione di un classificatore automatico simula il fenomeno dell'apprendimento umano o animale, noto come **training** (addestramento). L'idea è **dedurre regole generali**, applicabili a record non ancora classificati, partendo dall'osservazione di esempi già noti e ben classificati.

Gli algoritmi e le tecniche di classificazione automatica sono numerosissimi. Tutti i metodi noti condividono uno **schema generale** (Figura 3.1) ben testato e riconosciuto dalla comunità scientifica.

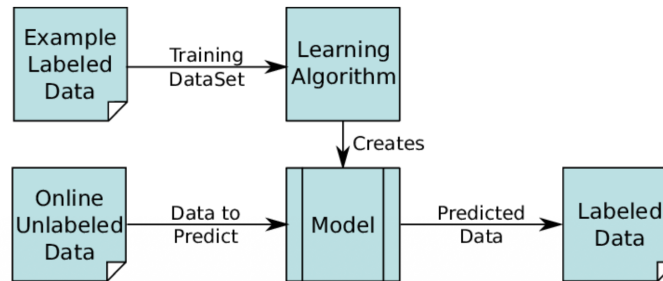


Figura 3.1: Schema generale di un sistema di classificazione

Definizione 3.2.1: Universo delle osservazioni

Si definisce **universo delle osservazioni** l'insieme complessivo dei record (passati, presenti e futuri) relativi ad un fenomeno. Molti algoritmi iniziano esaminando un sottoinsieme di questo universo, già classificato e ben compreso.

Un universo delle osservazioni (??), o un suo campione, che rappresenta l'insieme di “allenamento”, chiamato **Training Set (TS)**, è il deposito di informazioni iniziali da cui l'algoritmo ricava le “regole” di classificazione. Le regole ricavate saranno di vario tipo: statistiche, probabilistiche, fuzzy, funzioni discendenti, ecc.

3.2.1 Proprietà di un classificatore

Un buon insieme di regole di classificazione deve avere tre importanti proprietà:

- **Semplicità:** Le regole non devono essere troppo complicate, per garantire efficienza e basso costo computazionale in fase di classificazione.
- **Correttezza sul TS:** Le regole devono essere statisticamente sufficientemente corrette quando applicate al medesimo Training Set che le ha generate.
- **Generalizzabilità:** Le regole devono essere statisticamente corrette anche quando applicate al resto dei record dell'universo (dati nuovi, non visti).

Nota 3.2.1

Il termine **statisticamente corretto** su un training set indica che il tasso dei miss non deve superare certe soglie di tolleranza che dipendono dalla criticità delle applicazioni.

3.2.2 Il problema dell'overfitting

Le proprietà di correttezza sul TS e di generalizzabilità sono spesso in conflitto tra loro. Questo paradosso è noto come **overfitting** (sovradattamento). L'overfitting si verifica quando un modello si adatta “troppo bene” ai dati del Training Set. Un modello molto complesso può imparare a memoria le peculiarità e persino il rumore casuale presente nel TS, ottenendo una correttezza perfetta su di esso. Tuttavia, tale modello non avrà appreso la “tendenza” generale dei dati e fallirà nel generalizzare a nuovi record, poiché la frontiera di decisione che ha appreso è eccessivamente complessa e specifica per il campione di training.

L'obiettivo non è minimizzare l'errore sul TS (che porterebbe a un modello complesso e in overfitting), ma trovare un equilibrio: un modello (es. una retta o una curva semplice) che, pur commettendo qualche errore sul TS, catturi la struttura di fondo dei dati e possa quindi generalizzare meglio.

3.2.3 Validazione

Per “convalidare” la proprietà di generalizzazione di un insieme di regole, si utilizza un metodo che prevede, oltre al TS, un altro insieme di record già etichettati, detto **Control Set (CS)** o **Test Set**. Il CS **non** viene utilizzato durante la fase di training (cioè per la sintesi delle regole). Viene usato solo dopo che le regole sono state definite. Se le regole mostrano sul CS un tasso di errore (miss) simile a quello ottenuto sul TS, allora si ritiene che le regole siano **generalizzabili**.

Poiché anche il CS è un campione casuale, per una stima più precisa è buona norma ripetere i test con diversi CS, spesso creati tramite strategie di randomizzazione nella selezione del TS e del CS dall'universo disponibile.

3.3 Gestione delle feature e del rumore

Nella costruzione di un classificatore è cruciale gestire sia la selezione delle features (Definizione 3.1.1) che la presenza di rumore.

3.3.1 Selezione delle feature

Spesso si rilevano molte features (Definizione 3.1.1), ma non tutte sono utili; alcune possono essere sovrabbondanti o addirittura dannose. Combinare più features (Definizione 3.1.1) (es. lunghezza e luminosità dei pesci) è spesso una strategia conveniente, ma non è detto che sia sempre la migliore. L'inclusione di troppe features (Definizione 3.1.1), specialmente se irrilevanti, può amplificare il rumore e confondere il classificatore. Una buona pratica è scegliere features (Definizione 3.1.1) che siano **invarianti** alle trasformazioni tipiche della situazione sperimentale (es. il peso di un pesce è invariante alle condizioni di luce, la luminanza no). Inoltre, deve esistere una probabile relazione tra le features (Definizione 3.1.1) misurate e le classi (Definizione 3.1.2) da predire.

3.3.2 Rumore e outlier

I dati del mondo reale contengono inevitabilmente **rumore**, ovvero perturbazioni dovute a fenomeni non controllabili o non noti. Le cause di tale spostamento dai valori “ideali” possono essere:

- **Endogene**: Interne al fenomeno (es. un pesce con una dieta o storia anomala).
- **Esogene**: Dovute all'osservatore o allo strumento utilizzato (es. macchina fotografica starata, etichettatore distratto).

I dati molto “fuori norma” rispetto ai valori tipici di una classe (Definizione 3.1.2) sono definiti **outlier**. Un buon algoritmo di classificazione deve essere **robusto**, ovvero deve avere una forma di “protezione” o resistenza alle deviazioni che il rumore impone al processo decisionale.

3.4 Valutazione degli errori

Contare gli errori è essenziale, ma una singola percentuale di errore non è sufficientemente descrittiva. Questo perché non tutti gli errori sono uguali: i **costi degli errori** spesso **non sono uniformi** o simmetrici.

Ad esempio, in una diagnosi medica:

- Classificare un sano come malato (Falso Positivo) è un errore con un costo relativamente basso (paura, un test aggiuntivo).
- Classificare un malato come sano (Falso Negativo) è un errore gravissimo, che ritarda la diagnosi e può costare la vita al paziente.

Per analizzare questa asimmetria si usa la **matrice di confusione**. È una griglia quadrata che riporta quante istanze della classe (Definizione 3.1.2) “reale” (sulle colonne) sono state assegnate alla classe (Definizione 3.1.2) “prevista” (sulle righe).

Nota 3.4.1: Matrice di confusione ideale

Un classificatore perfetto ha come matrice di confusione la matrice identica (tutti i valori sulla diagonale principale, zero altrove). Un buon classificatore avrà valori percentuali bassi al di fuori della diagonale principale.

3.5 Fasi di un sistema di classificazione

Il processo di classificazione automatica si articola in diverse fasi:

1. **Sensing (o sampling):** Raccolta dei dati dal mondo fisico e loro digitalizzazione.
2. **Segmentazione:** Partizione dei dati in unità significative, pulizia ed eliminazione di dati irrilevanti.
3. **Estrazione delle feature:** Misurazione delle caratteristiche (quantitative o qualitative). È cruciale scegliere Definizione [3.1.1](#) invarianti e rilevanti.
4. **Classificazione:** Esecuzione dell'algoritmo scelto per l'assegnazione delle Definizione [3.1.3](#).
5. **Post-processing:** Valutazione della qualità della classificazione e dei costi associati agli errori.
6. **Decisione:** Utilizzo effettivo del classificatore per risolvere il problema reale.

La costruzione di un classificatore è un **ciclo iterativo** che prevede la raccolta dei dati, la selezione delle feature, la scelta di un modello matematico, il training dell'algoritmo e infine la sua valutazione, ripetendo i passi per migliorare le prestazioni.

3.6 Tecniche di validazione per la classificazione

Dopo aver costruito un modello di classificazione, è fondamentale valutarne le performance. A differenza della regressione, dove si cerca di predire un valore numerico di output dato uno o più valori di input, nella classificazione si vuole predire la Definizione [3.1.2](#) di un oggetto dato uno o più dati di input di qualsiasi tipo (numerici, categorici, testuali, ecc.). Per questo motivo gli strumenti che si possono applicare per valutare un modello di classificazione sono sostanzialmente diversi rispetto alle metriche utilizzate per valutare i modelli di regressione.

3.7 Modello di validazione base: Training e Test Set

L'approccio base per la validazione consiste nel dividere, secondo una certa **percentuale** i dati disponibili in due insiemi:

- **Training set (TS):** Utilizzato per addestrare il classificatore. Le Definizione [3.1.3](#) di questi dati sono usate per addestrare il classificatore.
- **Test set:** Utilizzato per valutare la bontà del modello. Le Definizione [3.1.3](#) di questo set vengono usate solo per verificare se il classificatore ha predetto correttamente.

L'obiettivo della validazione non è solo misurare l'errore, ma ha lo scopo di rispondere a domande più complesse, come:

- Il classificatore performa in modo bilanciato su tutte le Definizione [3.1.2](#)?
- Ha delle preferenze?
- Tali preferenze da cosa dipendono?

Per questo motivo per la valutazione dei classificatori si utilizzano indici matematici che permettono sia di avere stime oggettive delle performance, sia di automatizzare anche altre fasi del processo della progettazione o sviluppo del classificatore.

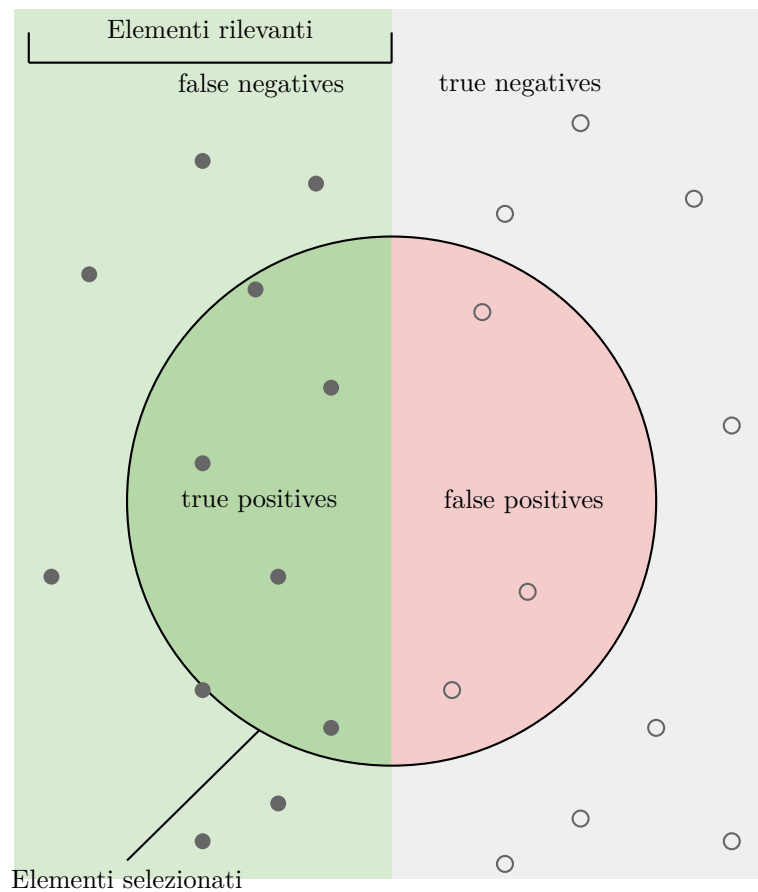
Gli indici principali relativamente all'aspetto computazionale utilizzati per valutare un classificatore sono:

- **Accuratezza:** La bontà nel predire correttamente le Definizione [3.1.3](#).
- **Robustezza:** La capacità di gestire dati con rumore o valori mancanti.
- **Velocità:** Include sia il tempo per costruire il modello (training time) sia il tempo per usarlo (classification/prediction time).
- **Scalabilità:** L'efficienza del modello su grandi dataset, specialmente se in memoria secondaria.
- **Interpretabilità:** La facilità con cui i risultati del modello possono essere compresi.

3.8 Metriche di valutazione

Per definire le metriche più comuni, si assume un problema di classificazione binaria. Si assume che l'insieme delle classi $\mathbb{C}$ dell'esperimento sia composto da due classi: $\mathbb{C} = \{A, B\}$. Relativamente ad una delle classi è possibile definire alcune misure per calcolare la bontà dell'algoritmo in valutare tale classe. Data una classe di interesse (es. A, la classe “positiva”), i risultati della classificazione sul test set vengono divisi in quattro categorie:

- True positive (TP).
- True negative (TN).
- False positive (FP).
- False negative (FN).



Definizione 3.8.1: TP, TN, FP, FN

Sia $c : CS \mapsto \mathbb{C}$ la funzione che mappa ogni record $x \in CS$ nella sua Definizione 3.1.2 reale e sia $\tilde{c} : CS \mapsto \mathbb{C}$ il classificatore che assegna una Definizione 3.1.2 ad A .

Sia $C = \{A, B\}$ l'insieme delle Definizione 3.1.2, composto dalle Definizione 3.1.2 A e B . Prendendo come riferimento la Definizione 3.1.2 A è possibile dividere CS in 4 insiemi:

- **True positive (TP)**: I record $x \in CS$ classificati **correttamente**, ovvero la cui Definizione 3.1.2 reale è A , quindi $\tilde{c}(x) = c(x) = A$
- **True negative (TN)**: I record $x \in CS$ classificati **correttamente**, ovvero la cui Definizione 3.1.2 reale è B , quindi $\tilde{c}(x) = c(x) = B$
- **False positive (FP)**: I record $x \in CS$ classificati **erroneamente**, ovvero la cui Definizione 3.1.2 reale è B , quindi $\tilde{c}(x) = B \neq A = c(x)$
- **False negative (FN)**: I record $x \in CS$ classificati **erroneamente**, ovvero la cui Definizione 3.1.2 reale è A , quindi $\tilde{c}(x) = A \neq B = c(x)$

Basandosi su queste quattro categorie, è possibile definire le metriche di performance più utilizzate.

Definizione 3.8.2: Precision

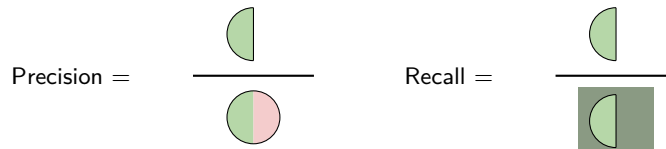
Sia $C = \{A, B\}$ l'insieme delle Definizione 3.1.2, composto dalle Definizione 3.1.2 A e B . La precision (precisione) è la frazione di elementi rilevanti per una Definizione 3.1.2 di riferimento, A , tra tutti gli elementi che il classificatore ha identificato come A . La precision misura quanto è "affidabile" la predizione positiva, ed è definita come:

$$Precision = \frac{TP}{TP + FP}$$

Definizione 3.8.3: Recall

Sia $C = \{A, B\}$ l'insieme delle Definizione 3.1.2, composto dalle Definizione 3.1.2 A e B . La recall (richiamo o sensitività) è la frazione di elementi rilevanti (Definizione 3.1.2 A) che sono stati correttamente classificati come A . Misura la capacità del classificatore di "trovare" tutti i positivi.

$$Recall = \frac{TP}{TP + FN}$$

Rappresentazione grafica delle metriche

Nota 3.8.1: Valori ottenuti

Valori alti per entrambe le metriche indicano un buon classificatore. Spesso, però, si preferisce utilizzare un indice unico che le combini.

Definizione 3.8.4: F_1 -Score

Il F_1 -Score rappresenta la media armonica di precision e recall. Fornisce un equilibrio tra le due metriche.

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Come precision e recall, anche l' F_1 -Score ha un valore compreso tra 0 e 1. Maggiore è il valore, maggiore è la bontà del classificatore.

Nota 3.8.2: Overfitting

Sebbene un F_1 -Score alto sia desiderabile, valori molto prossimi a 1 possono essere un campanello d'allarme per l'overfitting.

3.9 Accuracy

Definizione 3.9.1: Accuracy

La accuracy (accuratezza) misura la quantità totale di oggetti classificati correttamente (sia positivi che negativi) rispetto al totale degli oggetti.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Nota 3.9.1: Accuratezza per dataset sbilanciati

L'accuracy standard è poco indicata se le classi non sono bilanciate. Ad esempio, in un dataset con 95 campioni negativi e 5 positivi, un classificatore “pigro” che predice sempre “negativo” otterrebbe un'accuratezza del 95%, pur essendo totalmente inutile nel riconoscere i positivi.

In situazione di sbilanciamento delle classi, si preferisce la Balanced accuracy.

Definizione 3.9.2: Balanced accuracy

La Balanced accuracy (accuratezza bilanciata) è la media tra la sensitività (per i positivi) e la specificità (per i negativi).

$$Balanced\ accuracy = \frac{TPR + TNR}{2}$$

dove:

- **TPR (True Positive Rate):** È la Recall/Sensitività: $TPR = \frac{TP}{TP+FN}$.
- **TNR (True Negative Rate):** È la Specificità: $TNR = \frac{TN}{TN+FP}$.

3.10 Altri indici e matrice di confusione

Definizione 3.10.1: False Discovery Rate (FDR)

Misura il tasso di errori di tipo I (“false scoperte” o Falsi positivi) rispetto a tutte le predizioni positive.

$$FDR = \frac{FP}{FP + TP} = 1 - Precision$$

Matrice di confusione

La matrice di confusione è una tabella che riassume le performance di un classificatore binario, incrociando le classi reali con quelle predette e mostrando i conteggi di TP, TN, FP e FN. È fondamentale perché non tutti gli errori hanno lo stesso costo, come discusso in precedenza (es. diagnosi medica errata).

		Actual class		
		Positive	Negative	
Predicted class	Positive	TP: True Positive	FP: False Positive (Type I Error)	Precision: TP ----- (TP + FP)
	Negative	FN: False Negative (Type II Error)	TN: True Negative	Negative Predictive Value: TN ----- (TN+FN)
		Recall or Sensitivity: TP ----- (TP + FN)	Specificity: TN ----- (TN + FP)	Accuracy: TP + TN ----- (TP + TN + FP + FN)

Figura 3.2: Esempio di matrice di confusione per classificazione binaria.

È importante notare che metriche come sensibilità, precisione e specificità dipendono dalla Definizione 3.1.2 presa in considerazione, mentre l'accuratezza è un indice globale.

3.11 Area Under the Curve (AUC) e curva ROC

L'AUC (Area Under the Curve) è una misura basata sulla curva ROC (Receiver Operating Characteristics).

Definizione 3.11.1: Curva ROC

Una curva ROC è un grafico che mostra le performance di un classificatore al variare di un suo parametro (es. una soglia). Mette in relazione il **True Positive Rate (TPR)** (sull'asse Y) con il **False Positive Rate (FPR)** (sull'asse X).

($FPR = 1 - Specificità = \frac{FP}{FP+TN}$).

Le curve ROC passano sempre per i punti (0,0) e (1,1). Esistono inoltre due condizioni limite che rappresentano due curve di riferimento:

- Una retta che taglia il grafico a 45 gradi passando per l'origine. Questa retta rappresenta il caso del **classificatore casuale** e l'area sottesa (AUC) è pari a 0.5.
- Una curva rappresentata dal segmento che dall'origine sale verticalmente al punto (0,1) e dal segmento che congiunge il punto (0,1) a (1,1). Questa curva ha un'area sottesa di valore pari a 1 e rappresenta il **classificatore perfetto**.

L'AUC, ha un valore compreso tra 0 e 1, e misura l'intera area bidimensionale sotto la curva ROC.

- **AUC = 1**: Rappresenta il classificatore perfetto, che passa per il punto (0,1)
- **AUC = 0.5**: Rappresenta il classificatore casuale (la linea diagonale).
- **AUC = 0**: Rappresenta il classificatore "perfettamente sbagliato" (che inverte tutte le predizioni).

Il valore di AUC (tra 0 e 1) può essere interpretato come la probabilità che il classificatore assegni un punteggio più alto a un individuo positivo scelto a caso, rispetto a un individuo negativo scelto a caso.

3.12 Classificazione multi-classe

Le misure viste finora (Precision, Recall, F_1 -score) sono definite per la classificazione binaria. Per applicarle a problemi con $K > 2$ Definizione 3.1.2, si perde la visione di performance globale. Per l' F_1 -score, si possono calcolare delle medie. L'approccio comune è “one-vs-rest”: per ogni Definizione 3.1.2 $g_i \in G = \{1, \dots, K\}$, si costruisce una matrice di confusione dove g_i è il “caso positivo” e tutte le altre Definizione 3.1.2 formano il “caso negativo”. Si calcolano così TP_i , FP_i e FN_i per ogni Definizione 3.1.2 i .

Micro-average

La micro-average (micro-media) aggrega i contributi di tutte le Definizione 3.1.2 “sull’unità più piccola” (i singoli campioni) prima di calcolare le metriche. Queste metriche sono:

$$P_{micro} = \frac{\sum_{i=1}^{|G|} TP_i}{\sum_{i=1}^{|G|} (TP_i + FP_i)}$$

$$R_{micro} = \frac{\sum_{i=1}^{|G|} TP_i}{\sum_{i=1}^{|G|} (TP_i + FN_i)}$$

Da cui si può derivare il F_1 -score micro-averaged, $F_{1_{micro}}$ che rappresenta la media armonica di P_{micro} e R_{micro} .

$$F_{1_{micro}} = 2 \cdot \frac{P_{micro} \cdot R_{micro}}{P_{micro} + R_{micro}}$$

Nota 3.12.1: Micro-average e classi sbilanciate

Questa misura non è sensibile alle prestazioni sulle singole classi e può essere fuorviante se la distribuzione delle classi è sbilanciata.

Macro-average

La macro-average (macro-media) calcola la media su gruppi più vasti.

$$P_{macro} = \frac{\sum_{i=1}^{|G|} P_i}{|G|}$$

$$R_{macro} = \frac{\sum_{i=1}^{|G|} R_i}{|G|}$$

Da cui si può derivare il F_1 -score macro-averaged, $F_{1_{macro}}$ che rappresenta la media armonica di P_{macro} e R_{macro} .

$$F_{1_{macro}} = 2 \cdot \frac{P_{macro} \cdot R_{macro}}{P_{macro} + R_{macro}}$$

Nota 3.12.2: Macro-average per dati sbilanciati

Se questo valore è grande, indica che il classificatore funziona bene (in media) per ogni singola Definizione 3.1.2. Per questo motivo è più adatto per dati con distribuzione sbilanciata.

Generalizzazione di AUC (Metodo Hand & Till)

Esiste anche una generalizzazione dell'AUC per $k > 2$ Definizione 3.1.2 (Metodo Hand & Till, 2001). L'idea è calcolare una misura di separabilità $\hat{A}(i|j)$ per ogni possibile coppia di Definizione 3.1.2 (i, j) .

Definizione 3.12.1: Generalizzazione AUC

Sia $\hat{A}(i|j)$ la probabilità che dato un elemento a caso della Definizione 3.1.2 j abbia probabilità inferiore di attribuire quell'elemento alla Definizione 3.1.2 i , rispetto al valore di probabilità che attribuirebbe ad un elemento a caso della Definizione 3.1.2 i . È possibile calcolare $\hat{A}(i|j)$ utilizzando le seguenti definizioni:

- $\hat{p}(X_l)$ è la probabilità stimata che l'osservazione l sia originata dalla Definizione 3.1.2 i .
- per tutte le osservazioni x_l della Definizione 3.1.2 i , sia $f_l = \hat{p}(X_l)$. la probabilità stimata di appartenere alla Definizione 3.1.2 i .
- per tutte le osservazioni x_k della Definizione 3.1.2 j , sia $g_k = \hat{p}(X_k)$. la probabilità stimata di appartenere alla Definizione 3.1.2 i .

Allora i valori ottenuti ordinati in modo crescente sono: $\{g_1, \dots, g_n, f_1, \dots, f_n\}$. Sia r_l il rango della l -esima osservazione della classe i .

Il numero totale di coppie di punti in cui il punto della classe j ha un valore di probabilità stimato di appartenenza alla classe i inferiore a quello della classe i è:

$$\sum_{l=1}^{N_i} (r_l - l) = \sum_{l=1}^{N_i} r_l - \sum_{l=1}^{N_i} l = S_i - \frac{N_i(N_i + 1)}{2}$$

Dove N_i e N_j sono il numero di osservazioni delle classi i e j e S_i è la somma dei ranghi delle osservazioni della classe i .

La probabilità che un punto scelto a caso della classe j abbia una probabilità stimata di appartenenza alla classe i inferiore a quella di un punto scelto a caso della classe i è quindi:

$$\hat{A}(i|j) = \frac{S_i - \frac{N_i(N_i+1)}{2}}{N_i \cdot N_j}$$

Inoltre considerando che non è possibile distinguere $\hat{A}(i|j)$ da $\hat{A}(j|i)$, si ha che la misura di separabilità tra le classi i e j è data dalla media tra $\hat{A}(i|j)$ e $\hat{A}(j|i)$, ovvero:

$$\hat{A}(i|j) = \frac{\hat{A}(i|j) + \hat{A}(j|i)}{2}$$

Il valore di AUC globale (M) di un classificatore multi-classe è quindi dato dalla media di tutti i valori $\hat{A}(i|j)$ calcolati è definito come:

$$M = \frac{2}{c(c-1)} \sum_{i < j} \hat{A}(i|j)$$

Dove c è il numero totale di classi, $\frac{2}{c(c-1)}$ è un fattore che viene applicato perchè sono presenti $c(c-1)$ modi differenti con cui costruire coppie distinte di classi.

3.13 Cross-validation

Definizione 3.13.1: Cross-Validazione

È una tecnica statistica usata per validare un modello e valutare come i suoi risultati si generalizzeranno a un insieme di dati indipendente. L'obiettivo primario è testare la capacità del modello di prevedere su nuovi dati, non usati durante l'addestramento. Serve principalmente a stimare problemi di **overfitting** o di **selection bias**.

Il *selection bias* si verifica quando la scelta del training set è viziata (da fattori esterni) e non rispecchia un campionamento uniforme dell'universo delle osservazioni.

Nota 3.13.1: Selection bias

Il *selection bias* può portare a stime distorte delle performance del modello, poiché il training set non rappresenta adeguatamente la popolazione generale. È importante essere consapevoli di questo bias durante la fase di progettazione dello studio e nella raccolta dei dati. Training set e test set dovrebbero essere prodotti tramite campionamento uniforme dell'universo delle possibili osservazioni.

La cross-validazione si divide in due tipi principali:

- Cross-validazione esaustiva, che testa tutte le possibili divisioni del dataset in TS e CS.
- Cross-validazione non esaustiva, che testa solo un sottoinsieme delle possibili divisioni.

Cross-validazione esaustiva

La cross-validazione esaustiva testa tutti i modi possibili di dividere il dataset in TS e CS.

- **Leave-p-out (LPO)**: Utilizza p osservazioni come CS e $N - p$ come TS. Questo processo viene ripetuto per tutti le $\binom{n}{p}$ possibili combinazioni.
- **Leave-one-out (LOOCV)**: È un caso particolare di LPO dove $p = 1$. È appropriata per dataset molto piccoli, dove il costo computazionale è secondario rispetto all'accuratezza della stima.

Cross-validazione non esaustiva

La cross-validazione non esaustiva testa solo un sottoinsieme delle possibili divisioni. La tecnica più comune è la **k-fold cross-validation**, dove il dataset viene diviso casualmente in k parti (fold) di eguale dimensione. A turno, ogni "fold" viene usato come Test Set (CS) e i restanti $k - 1$ fold vengono usati come Training Set (TS). Il processo si ripete k volte e le metriche vengono mediate.

Capitolo 4

Algoritmi di classificazione

4.1 Tipologie di modelli di classificazione

I modelli di classificazione possono essere suddivisi in due tipologie:

- **Eager (volenterosi):** Costruiscono un modello generale e indipendente dall'input a partire dai dati di addestramento. Tale modello viene poi usato per classificare nuovi dati.
- **Lazy (pigri):** Memorizzano semplicemente i dati di addestramento (in forma grezza o con piccole elaborazioni) e rimandano l'intero processo di classificazione all'arrivo della nuova istanza da classificare.

Nota 4.1.1: Tempi di addestramento e classificazione

I metodi lazy richiedono meno tempo in fase di addestramento ma molto più tempo in fase di classificazione.

Nota 4.1.2: Accuratezza e spazio delle ipotesi

In termini di accuratezza i metodi lazy utilizzano uno spazio delle ipotesi più ricco, avendo la possibilità di usare funzioni locali di classificazione che forniscono approssimazioni a livello globale. Al contrario, i metodi eager si riferiscono a una sola ipotesi per coprire l'intero universo delle osservazioni.

Approcci tipicamente lazy includono:

- **k-nearest neighbors (kNN)** (i k vicini più prossimi): I data point sono rappresentati in uno spazio euclideo.
- **Locally weighted regression:** Costruisce approssimazioni locali.

Approcci tipicamente eager includono:

- **Alberi di decisione:** Costruisce alberi di ricerca basati su test sequenziali delle feature.

- **Support Vector Machines (SVM):** Determinano le superfici decisionali (iperpiani) che separano le classi con il maggior margine possibile.
- **Metodi do consenso (Ensemble Methods):** Combinano molteplici modelli per migliorare l'accuratezza e la stabilità rispetto a un singolo classificatore.
- **Analisi discriminante:** Utilizza il teorema di Bayes per stimare la probabilità di appartenenza a una classe assumendo che i predittori seguano una distribuzione normale.

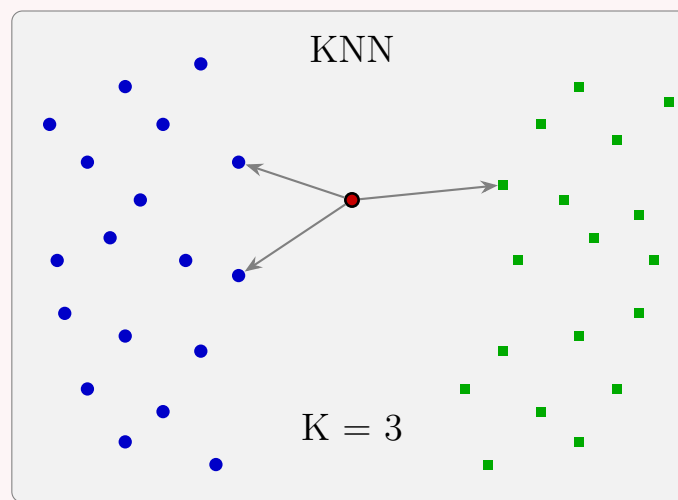
4.2 Approcci lazy

k-Nearest Neighbors (kNN)

Nel kNN l'insieme delle osservazioni è rappresentato come un insieme di data point in uno spazio n -dimensionale, dove ogni dimensione può avere valori reali o discreti. Lo spazio è tipicamente euclideo (su cui si calcolano distanze euclidee), ma è possibile utilizzare anche feature categoriali con metriche diverse (es. Jaccard, Tanimoto, Dice, Hamming, distanza al coseno).

Definizione 4.2.1: k-Nearest Neighbors (kNN)

ato un training set (TS) e un'istanza da classificare, l'algoritmo assegna a tale istanza la classe che è più rappresentata tra i suoi k vicini più prossimi.



Nota 4.2.1: Il ruolo di k

Il parametro k è un punto chiave dell'algoritmo perché determina il numero di vicini da considerare per la classificazione.

Il kNN è uno strumento **non parametrico**, cioè non fa alcuna ipotesi sulla distribuzione dei dati. Questo è utile perché la maggior parte dei dati “reali” non obbedisce ad assunti teorici (come nei modelli di regressione lineare).

Di conseguenza, kNN è spesso una delle prime scelte per uno studio di classificazione quando c'è poca o nessuna conoscenza sulla distribuzione dei dati.

Potere predittivo e scelta di k Il potere predittivo dipende dal parametro k :

- **k piccolo:** Limita la regione di previsione, rendendo il classificatore “più cieco” alla distribuzione generale.

- **k grande:** Riduce l'impatto della varianza e dell'errore casuale, ma rischia di ignorare piccoli dettagli rilevanti.

Alcuni autori suggeriscono di impostare k uguale alla radice quadrata del numero di osservazioni nel training set. Un'alternativa migliore è usare la cross-validazione per identificare il valore di k con le performance migliori.

Weighted kNN

Per migliorare le performance, si può usare una versione pesata (Weighted kNN), dove le classi dei vicini sono pesate in base alla loro distanza. Ad esempio, si può pesare l'apporto di ogni vicino con l'inverso della sua distanza ($1/d_i$).

Nota 4.2.2: Weighted kNN e dataset sbilanciati

Per dataset sbilanciati, si può anche pesare l'apporto di ogni vicino con l'inverso del numero totale di punti nel dataset aventi la stessa classe del vicino in questione.

4.3 Approcci eager

Alberi di decisione

Definizione 4.3.1: Albero di decisione

Un albero di decisione descrive un albero di ricerca (n-ario) dove i nodi foglia rappresentano le classificazioni e le ramificazioni l'insieme delle proprietà che portano a quelle classificazioni. Ogni nodo interno contiene un test (tipicamente su una feature o una combinazione di feature) che stabilisce quale sotto-albero deve essere visitato.

Costruzione dell'albero L'algoritmo di costruzione (top-down, greedy) inizia con un nodo radice che contiene tutti i dati di addestramento e segue una strategia ricorsiva:

1. Se tutti i dati di training in un nodo t hanno lo stesso valore sull'attributo classificatore, si crea un nodo foglia contenente tutti i dati.
2. Altrimenti, si cerca la partizione S degli elementi (basata su un attributo) che massimizza una misura di "goodness" (purezza).
3. Si sceglie la partizione S^* che massimizza la misura di "goodness".
4. Si creano tanti figli del nodo t quante sono le classi/valori presenti in S^* .
5. Un nodo figlio è detto "puro" se tutti i suoi elementi hanno lo stesso valore di classe.
6. Si applica ricorsivamente l'algoritmo sui nodi impuri (o che non rappresentano una singola classe secondo una soglia di maggioranza).

L'obiettivo è scegliere l'albero più semplice a parità di performance e il più compatto possibile. Trovare l'ipotesi minimale è NP-hard, per cui si usa questa strategia greedy (divide-et-impera) che non garantisce l'ottimalità.

La decisione principale è scegliere l'attributo su cui biforcare in modo che divida in insiemi il più puri possibile e porti rapidamente a nodi foglia.

Goodness functions Le misure di "goodness" (purezza) più utilizzate per scegliere l'attributo migliore per lo split sono:

- **Information Gain (ID3, C4.5):** Basato sulla riduzione dell'entropia. Adatto per attributi categoriali (ma modificabile per i continui).
- **Indice di Gini (CART):** Adatto per attributi continui, assume che esistano diversi valori di splitting per ogni attributo (tipicamente split binari).

Information gain La scelta basata sull'**information gain** mira alla riduzione progressiva dell'entropia. L'entropia H misura il disordine (o l'incertezza media) di una variabile aleatoria.

$$H(p_1, \dots, p_k) = - \sum_i p_i \log_b p_i$$

L'entropia è massima quando tutti gli eventi sono equiprobabili (massimo disordine).

Dato un insieme S con k classi C_i , l'informazione (entropia) necessaria per classificare un campione in S misura la **quantità di informazione media necessaria** ed è definita come:

$$Info(S) = - \sum_{i=1}^k \frac{freq(C_i, S)}{|S|} \log_2 \frac{freq(C_i, S)}{|S|}$$

Se si partiziona S usando un attributo A (con n valori) nei sottoinsiemi $\{S_1, \dots, S_n\}$, l'entropia dell'albero con radice A è la media ponderata delle entropie dei sottoinsiemi:

$$Info_A(S) = \sum_{i=1}^n \frac{|S_i|}{|S|} Info(S_i)$$

La quantità di informazione guadagnata dal partizionamento di S è la riduzione dell'entropia ottenuta splittando sull'attributo A :

$$Gain(A) = Info(S) - Info_A(S)$$

L'algoritmo seleziona l'attributo A che massimizza la $Gain(A)$ (o, equivalentemente, minimizza $Info_A(S)$).

Esempio 4.3.1: Information gain con due classi

ia C l'insieme delle classi con due classi C_1 e C_2 , e sia S un insieme che contiene c_1 elementi di C_1 e c_2 elementi di C_2 .

La **quantità di informazione** necessaria per decidere se un elemento arbitrario in S appartiene a C_1 o C_2 è:

$$Info(c_1, c_2) = - \frac{c_1}{c_1 + c_2} \log_2 \frac{c_1}{c_1 + c_2} - \frac{c_2}{c_1 + c_2} \log_2 \frac{c_2}{c_1 + c_2}$$

Assumendo di utilizzare un attributo A con v valori $\{a_1, a_2, \dots, a_v\}$, come radice dell'albero, allora S viene partizionato in v sottoinsiemi $\{S_1, S_2, \dots, S_v\}$.

Così facendo C_1 e C_2 si dividono in $\{[C_1]_1, [C_1]_2, \dots, [C_1]_v\}$ e $\{[C_2]_1, [C_2]_2, \dots, [C_2]_v\}$.

Se S_i contiene $[c_1]_i$ elementi di C_1 e $[c_2]_i$ elementi di C_2 , allora la quantità di informazione richiesta per classificare gli oggetti nei vari S_i saranno $Info([c_1]_i, [c_2]_i)$.

Di conseguenza l'informazione richiesta per l'albero con radice A sarà la media pesata dagli $Info$ secondo le partizioni che A impone su S :

$$Info_A(S) = \sum_{i=1}^v \frac{[c_1]_i + [c_2]_i}{c_1 + c_2} Info([c_1]_i, [c_2]_i)$$

Nota 4.3.1: Sbilanciamento dell'Information Gain

L'Information gain è fortemente sbilanciato in favore dei test che hanno molti esiti (es. un attributo ID), anche se questi esiti sono poco significativi in termini di predizione. Come soluzione a questa problematica si normalizza il gain in base alla quantità di informazione ottenibile da S stesso.

Considerando l'**informazione** contenuta da un messaggio sulla probabile partizione di S in S_i insiemi, si ha che tale informazione è data da:

$$-\log_2 \frac{|S_i|}{|S|}$$

Per analogia la definizione di $Info(S)$ è porta alla definizione di $SplitInfo$, che rappresenta la potenziale informazione generata dalla divisione di S .

$$SplitInfo(A) = - \sum_{i=1}^n \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

Il **gain normalizzato**, ottenuto usando $SplitInfo$, è quindi:

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

Il GainRatio non risolve tutti i problemi, infatti potrebbe succedere che attributi significativi assumono un qualche valore in più rispetto agli altri risultando svantaggiati.

Una strategia comune è:

1. Calcolare il $Gain(A)$ per ogni attributo.
2. Calcolare la media dei guadagni.
3. Selezionare **solo** gli attributi con guadagno sopra la media.
4. Scegliere, tra gli attributi selezionati, quello con $GainRatio$ maggiore.

Nota 4.3.2: Attributi continui

Per attributi di tipo continuo, si valuta l'utilizzo di un valore di soglia Z , che rende il test binario: $A > Z$ o $A \leq Z$.

Si testano tutti i possibili valori di soglia v_j (o i punti medi tra valori ordinati) presenti nel training set e si sceglie la soglia che massimizza il gain. Così facendo l'insieme di dati viene diviso in due sottoinsiemi $\{v_1, v_2, \dots, v_i\}$ e $\{v_{i+1}, v_{i+2}, \dots, v_m\}$.

Questo criterio va ad introdurre implicitamente un'altra restrizione, richiedendo che gli insiemi generati abbiano almeno un certo numero minimo di elementi.

Nota 4.3.3: Gestione dei valori mancanti

Durante la costruzione dell'albero, può capitare che alcuni record abbiano valori mancanti (nulli) per alcuni attributi. Tra le varie opzioni per gestire questi casi, si possono usare varie strategie:

- Assegnare il valore più comune per quell'attributo.
- Assegnare una probabilità a ognuno dei possibili valori.
- Attribuire un peso ai record.

In queste situazioni è necessario adattare il calcolo del $Gain(A)$:

$$Gain(A) = F \cdot (Info(S) - Info_A(S))$$

dove F è la percentuale di casi nel training set in cui l'attributo A è noto.

La variazione sul calcolo del $Gain(A)$ può essere estesa anche al calcolo della $SplitInfo(A)$, e di conseguenza del $GainRatio(A)$, influenzando così sul partizionamento.

Per pesare i singoli record, è necessario valutare che se un record ha un valore sconosciuto il peso è proprio la probabilità che il valore sia v_i . Ogni sottoinsieme S_i è una collezione di casi con pesi possibilmente frazionali, così che $|S_i|$ può essere reinterpretata come la somma dei pesi frazionali dei casi del sottoinsieme.

In generale, un caso di S con peso w il cui esito non è noto a ogni sotto insieme S_i con peso w^* ha la probabilità che l'esito sia v_i

Indice di Gini**Definizione 4.3.2: Indice di Gini**

L'indice di Gini è una metrica utilizzata per valutare la purezza dei nodi e guidare la costruzione degli alberi. Questa misura l'impurità di un insieme S .

Dato un insieme S con n classi, l'**indice di Gini** è definito come:

$$gini(S) = 1 - \sum_{j=1}^n p_j^2$$

dove p_j è la frequenza della classe j :

Definito questo è necessario adattare il calcolo dello split basato su Gini, definendo l'**indice di impurità dello split** come segue:

$$gini_{split}(S, A) = \sum_{i=1}^m \frac{N_i}{N} gini(S_i)$$

dove S è diviso in m sottoinsiemi S_i (di dimensione N_i) in base all'attributo A , e $N = \sum_{i=1}^m N_i$ è la dimensione totale di S .

Esempio 4.3.2: Indice di Gini con due classi

Date due classi P e N , tale che $S = P \cup N$, contenenti rispettivamente p e n elementi, è possibile calcolare:

$$p_p = \frac{p}{p+n}$$

$$p_n = \frac{n}{p+n}$$

$$gini(S) = 1 - p_p^2 - p_n^2$$

Da questo si può calcolare l'indice di Gini per lo split su un attributo A che divide S in due sottoinsiemi S_1 e S_2 , rispettivamente con dimensioni N_1 e N_2 .

$$gini_{split}(S, A) = \frac{N_1}{N} gini(S_1) + \frac{N_2}{N} gini(S_2)$$

Nota 4.3.4: Algoritmo di CART

L'algoritmo CART (che usa Gini) seleziona l'attributo a che ha il **minore** $gini_{split}(S, a)$, in quanto si cerca la massima riduzione dell'impurità. Per gli attributi numerici, CART testa tutti i possibili split binari.

Pruning Un albero cresciuto molto in profondità rischia di adattarsi troppo ai dati di training, inclusi gli outlier e il rumore. Questa situazione è detta **overfitting** e porta a un aumento degli errori su dati nuovi.

La soluzione è il **pruning** (potatura), ovvero la rimozione di parti dell'albero (rami) che non contribuiscono significativamente alla classificazione corretta, producendo un modello meno complesso e più generalizzabile.

Le strategie di pruning sono:

- **Pre-pruning:** Attuato durante la costruzione. Si decide di non dividere ulteriormente un nodo se il gain è inferiore a una soglia t , o se la purezza è già sufficiente (es. tramite metodi statistici).
- **Post-pruning:** Attuato dopo la costruzione completa dell'albero. Si recidono rami analizzando l'impatto sull'errore (spesso su un validation set). È più dispendioso ma generalmente più efficace (C4.5 usa il post-pruning).

Nota 4.3.5: Potature e distribuzione di probabilità

Quando si pota, la foglia risultante potrebbe non essere pura. Invece di una classe, le si associa una distribuzione di probabilità delle classi presenti.

C4.5 C4.5 (Pessimistic Pruning) utilizza un post-pruning “pessimistico” basato sul set di training stesso. Stima l'errore di un sottoalbero e lo confronta con l'errore medio che si otterrebbe sostituendo il sottoalbero con una singola foglia (o, in C4.5, anche con uno dei suoi rami). Se l'errore stimato del sottoalbero potato è minore, il nodo viene potato.

Minimal cost-complexity pruning CART (minimal cost-complexity pruning) è un algoritmo di potatura parametrizzato da $\alpha \geq 0$ (parametro di complessità).

Si definisce un costo di complessità $R_\alpha(T)$ per un albero T come:

$$R_\alpha(T) = R(T) + \alpha|\tilde{T}|$$

dove $R(T)$ è il tasso di errore (impurità) dell'albero e $|\tilde{T}|$ è il numero di nodi terminali (foglie). L'algoritmo cerca il sottoalbero che minimizza $R_\alpha(T)$.

Si calcola un *alpha* effettivo, α_{eff} come seguente:

- Se il nodo t è una foglia, $R_\alpha(T_t) = R(t)$.
- Altrimenti, per un nodo non terminale:

$$\alpha_{eff} = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Si pota iterativamente il nodo non terminale con il α_{eff} più piccolo. All'aumentare di α , l'albero viene potato di più, l'impurità (errore sul training) aumenta, ma la generalizzazione (errore sul test) migliora fino a un certo punto ottimale, per poi peggiorare (underfitting).

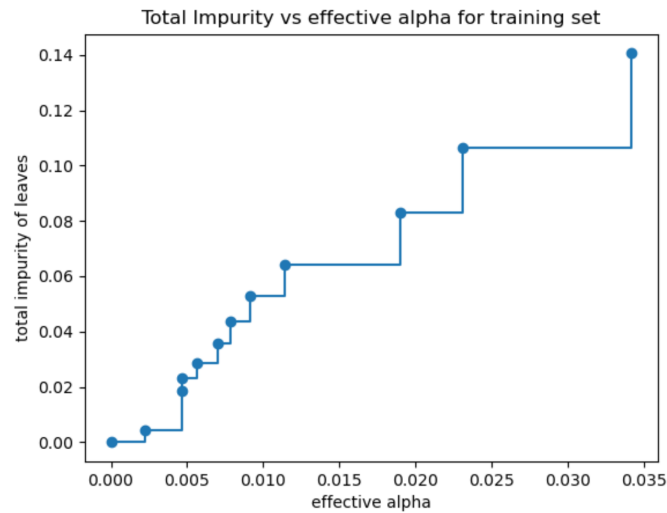


Figura 4.1: All'aumentare di α , viene potata una parte maggiore dell'albero, il che aumenta l'impurità totale delle sue foglie.

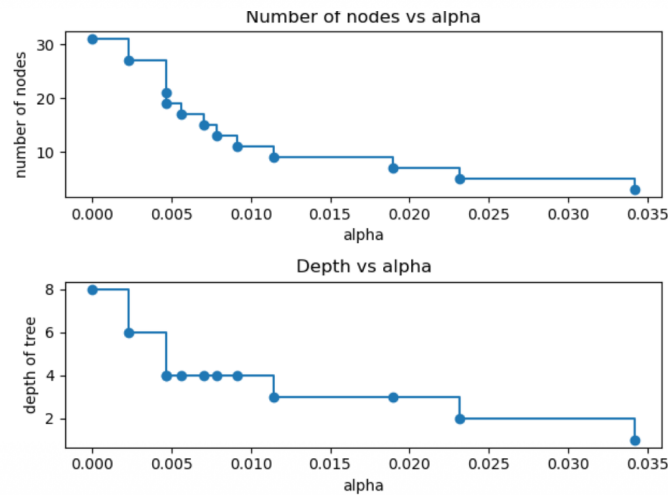


Figura 4.2: Il numero di nodi e la profondità dell'albero diminuiscono all'aumentare di α

Nota 4.3.6: Valore di α e overfitting

Quando il valore di soglia è impostato a 0 (nessuna potatura), l'albero va in overfitting, portando ad una precisione di allenamento del 100% ma ad una precisione di test dell' 80%.

All'aumento di α , viene tagliata una parte maggiore dell'albero, creando così un albero decisionale che generalizza meglio.

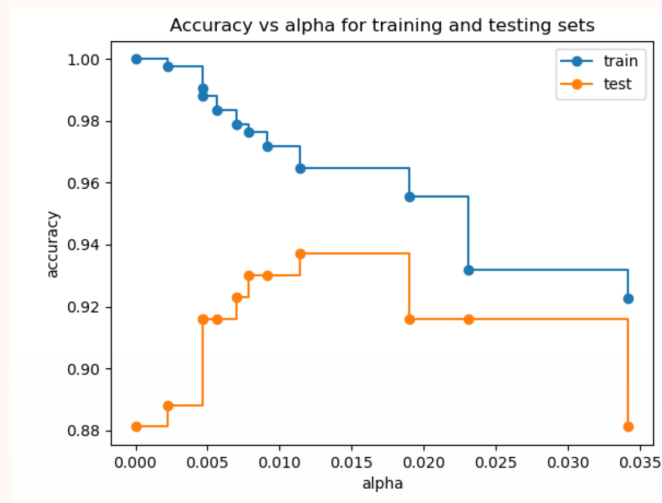


Figura 4.3: Andamento dell'accuratezza in funzione di α .

Metodi consenso

Gli alberi di decisione sono semplici e interpretabili, ma spesso non competitivi in termini di qualità predittiva. Inoltre, sono instabili, piccoli cambiamenti nei dati di training possono portare a un albero molto diverso (alta varianza).

Per ovviare a questi svantaggi, si utilizzano differenti strategie tra cui **bagging**, **random forests**, e **boosting**, che costruiscono molteplici alberi e li combinano in un'unica predizione **consenso**, migliorando notevolmente l'accuratezza a scapito (parziale) dell'interpretabilità.

L'obiettivo è impiegare una combinazione di modelli M^* , ottenuta combinando k modelli addestrati separatamente, per incrementare l'accuratezza. I metodi più popolari sono:

- **Bagging**: Media la predizione su una collezione di classificatori.
- **Boosting**: Impiega una collezione di classificatori tramite un voto pesato.
- **Ensemble**: Combina un insieme eterogeneo di classificatori.

Bagging Il bagging (Bootstrap Aggregation) è una tecnica generale per ridurre la varianza di un metodo statistico, particolarmente utile per gli alberi di decisione.

L'idea è simulare la disponibilità di più training set, anche se se ne ha uno solo:

1. Si estraggono B diversi sottoinsiemi (campioni **bootstrap**) dall'unico training set (estraendo casualmente con reinserimento).
2. Si apprende un classificatore (un albero) f_i per ciascun sottoinsieme i .
3. Si aggregano (ensemble) le predizioni sulle nuove istanze x di test:

$$f_{bag}(x) = \frac{1}{B} \sum_{i=1}^B f_i(x)$$

Random Forest Una Random Forest è un metodo consenso costituito da un **bagging di alberi di decisione non potati** (complessi), con un'aggiunta cruciale: **una scelta casuale di un sottoinsieme delle feature (predittori) ad ogni split**.

Nota 4.3.7: Metodologia

La random forest è un metodo non lineare (come gli alberi di decisione) e robusto, con bassa varianza e stabilità delle predizioni rispetto al variare dei dati di input.

Migliora le prestazioni di un singolo albero di decisione appreso su tutti i dati ma perde la facile interpretabilità e scala meno bene.

Addestramento della foresta Fissato un numero di alberi n_{tree} e il numero di predittori (numero di feature da testare a ogni split) m_{try} da scegliere casualmente tra quelle disponibili, viene ripetuto per n_{tree} la procedura seguente:

1. Si estrae un campione bootstrap S dai dati di training.

2. Si apprende un albero di decisione su S accurato ($\text{minsplit} = 1$), in cui ad ogni split solo m_{try} predittori sono scelti casualmente come candidati.
3. Non viene effettuato il pruning.
4. Si salva l'albero ottenuto tra quelli disponibili.

Predizione Si aggregano le predizioni degli n_{tree} alberi tramite voto di maggioranza (classificazione) o media (regressione).

Nota 4.3.8: Scelta di m_{try}

Di solito, $m_{try} \approx \sqrt{m}$ per la classificazione e $m_{try} \approx m/3$ per la regressione (dove m è il numero totale di feature).

Nota 4.3.9: Costruzione raffinata

Il bagging riduce la varianza mediando alberi non distorti (perché profondi) ma soggetti a errore (alta varianza). La varianza di una media di B variabili i.i.d. (indipendenti e identicamente distribuite) con varianza σ^2 è $\frac{1}{B}\sigma^2$.

Tuttavia, gli alberi del bagging non sono indipendenti, perché crescono dallo stesso TS (anche se bootstrapped). Sono correlati. Se la correlazione a coppie è $\rho > 0$, la varianza della media è:

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Al crescere di B , la varianza non va a zero, ma tende a $\rho\sigma^2$.

L'idea della Random Forest è **ridurre la correlazione** ρ tra gli alberi. Introducendo la selezione casuale di m_{try} feature ad ogni split, si “decorrelano” gli alberi. Questo permette di migliorare la riduzione della varianza del bagging, senza accrescere troppo la distorsione (bias).

Valutazione (OOB error): Le RF forniscono una **stima dell'errore** di test chiamata **Out-Of-Bag (OOB) error**. Per ogni istanza di training x_i , si usano per la predizione solo gli alberi che non contenevano x_i nel loro campione bootstrap (circa 1/3 degli alberi). L'OOB error è la media degli errori su queste predizioni.

Importanza delle features Si può stimare l'importanza di una feature in due modi:

- **Mean decrease in accuracy (permutation importance):** Si calcola l'accuratezza OOB. Poi, per una feature i , si permutano casualmente i suoi valori nei dati OOB (rompendo la correlazione con la classe). Si ricalcola l'accuratezza. La differenza media (su tutti gli alberi) nella precisione è l'importanza di quella feature.
- **Mean decrease in Gini index:** Si misura quanto lo split su una certa variabile riduce (in media) l'impurità (indice di Gini) attraverso tutti gli alberi.

Nota 4.3.10: Vantaggi

Le random forest (RF) presentano diversi vantaggi chiave rispetto ai singoli alberi decisionali:

- **Scalabilità:** La complessità temporale aumenta in modo contenuto, poiché ogni albero viene addestrato solo su un sottoinsieme di dati e, ad ogni split, viene considerato solo un sottoinsieme di predittori. Questo permette di gestire efficientemente anche dati di grandi dimensioni.
- **Resistenza all'Overfitting:** Le RF generalmente non soffrono di overfitting all'aumentare del numero di alberi. Ciò è dovuto alla combinazione della selezione casuale dei predittori a ogni split e all'aggregazione finale delle predizioni.
- **Stabilità:** Grazie all'uso del bagging, le RF sono più stabili rispetto alle variazioni nei dati di input.
- **Parallelizzabilità:** Gli alberi in una RF vengono appresi in modo indipendente (a differenza del boosting). Questo rende l'algoritmo facilmente parallelizzabile su hardware multi-core o multi-processore.

Nota 4.3.11: Svantaggi e limiti

Random forest non è ideale per:

- **Estrapolazione:** La regressione casuale della foresta non è l'ideale nell'estrapolazione dei dati (a differenza della regressione lineare, che utilizza le osservazioni esistenti per stimare i valori oltre l'intervallo di osservazione).
- **Scarsità dei dati:** Una RF non produce buoni risultati quando i dati sono molto scarsi (il sottoinsieme di funzionalità e il campione bootstrap produrranno uno spazio invariante, portando a divisioni improduttive che influenzeranno il risultato finale).

Boosting Il Boosting è un altro metodo consenso.

Nota 4.3.12: Idea del Boosting: analogia medica

L'idea del boosting può essere paragonata a quella di “consultare diversi medici, assegnando un peso alla loro diagnosi in base all'accuratezza delle diagnosi precedenti”.

L'idea è addestrare una serie di k classificatori (spesso alberi semplici, detti “weak learners”) in modo **sequenziale**:

1. Le tuple in D vengono campionate con “replacement” (rimettendo una coppia in D dopo l'estrazione) per formare un training set D_i della stessa dimensione di D .
2. La probabilità di ogni tupla di essere selezionato è proporzionale al suo peso.
3. Si addestra un classificatore M_i a partire da D_i .
4. Viene calcolato il tasso di errore di M_i utilizzando D_i come test set.

5. Le tuple classificate erroneamente vedono aumentare il loro peso, altrimenti (se classificate correttamente) il loro peso diminuisce.

Adaboost Adaboost (Adaptive Boosting) è l'algoritmo di boosting più utilizzato. Dato $err(X_j)$ l'errore nel classificare la tupla X_j (0 o 1 nei casi di classificazione e non di regressione). Con questo posso definire l'errore complessivo del classificatore M_i come:

$$err(M_i) = \sum_{j=1}^d w_j err(X_j)$$

dove w_j è il peso corrente della tupla X_j .

Il peso del classificatore M_i nella votazione consenso è definito come:

$$\log \frac{1 - error(M_i)}{error(M_i)}$$

Support vector machines

Le SVM (Macchine a vettori di supporto), introdotte da Vapnik (basate sulla statistical learning theory), sono uno strumento molto usato per la classificazione. Invece di stimare le densità di probabilità (come i classificatori Bayesiani), SVM cerca di determinare direttamente la **superficie decisionale** (classification boundary) tra le classi.

SVM nasce come classificatore binario e si estende a:

- SVM lineare (dati linearmente separabili).
- SVM lineare (dati non linearmente separabili, soft margin).
- SVM non lineare (kernel).
- Estensione multiclasse.

Date due classi linearmente separabili, esistono infiniti iperpiani in grado di separarle. SVM determina l'iperpiano che separa le due classi con il **maggior margine** possibile. Il **margine** è la distanza minima tra i punti delle due classi (nel training set) e l'iperpiano stesso.

Nota 4.3.13: Importanza del margine

La massimizzazione del margine è legata alla generalizzazione, se i dati di training sono classificati con ampio margine, si spera che anche i dati di test vicini al confine siano gestiti correttamente.

SVM Lineari (Dati Linearmente Separabili) Date due **classi linearmente separabili** e un training set TS contenente n campioni di coordinate $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, dove $x_i \in \mathbb{R}^d$ sono i **record multidimensionali** e $y_i \in \{+1, -1\}$ sono le etichette di classe, esistono diversi **iperpiani** in grado di eseguire la separazione voluta.

Un generico iperpiano è definito dai parametri (w, b) come (omettendo per semplicità il prodotto scalare):

$$D(x) = w \cdot x + b = 0$$

dove w è il vettore normale all'iperpiano e $\frac{b}{\|w\|}$ è la distanza dall'origine.

Capitolo 5

Clustering

5.1 Introduzione al clustering

5.2 Algoritmi di clustering

5.3 Valutazione del clustering

Capitolo 6

Grafi

6.1 Teoria dei grafi

6.2 Algoritmi di ricerca sui grafi

Capitolo 7

Algoritmi e metaeuristiche

7.1 Algoritmi di ricerca

7.2 Metaeuristiche

Capitolo 8

Teoria dei giochi

8.1 Introduzione

La teoria dei giochi estende il concetto di problem solving introducendo la nozione di agenti multipli in competizione. A differenza del planning, dove la complessità risiede nella gestione di stati (congiunzioni di fatti) e operatori (legami tra fatti-condizioni e fatti-effetti, non tra stati), nel game playing viene introdotto nel dominio del problema la competizione tra agenti.

Nota 8.1.1: Teoria della Decisione vs. Teoria dei Giochi

Il lavoro seminale di Von Neumann e Morgenstern (1944), *"Theory of Games and Economic Behaviour"*, distingue due aree:

- **Teoria della decisione:** Analizzare il comportamento di un agente le cui azioni hanno un effetto diretto, come nelle scommesse o nei puzzle.
- **Teoria dei giochi:** Analizza il comportamento di un agente le cui azioni hanno effetti che dipendono dalle scelte di altri agenti.

8.1.1 Classificazione dei giochi

I giochi possono essere classificati secondo diversi criteri.

Condizioni di scelta

- **Giochi a informazione perfetta:** Lo stato del gioco è completamente noto a tutti gli agenti (es. Scacchi, Go).
- **Giochi a informazione imperfetta:** Lo stato del gioco è solo parzialmente noto agli agenti (es. Poker, MasterMind).

Effetti della scelta

- **Giochi deterministici:** Lo stato successivo è determinato unicamente dall'azione compiuta (es. Scacchi).
- **Giochi stocastici:** Lo stato successivo dipende anche da fattori casuali (es. Backgammon, Monopoly, che usano dadi).

	Informazione Perfetta	Informazione Imperfetta
Giochi deterministici	Scacchi, Go, Dama, Otello, Forza4	MasterMind (è un gioco o un puzzle?)
Giochi stocastici	Backgammon, Monopoli	Scarabeo, Bridge, Poker... (giochi di carte) Risiko

Figura 8.1: Classificazione dei giochi secondo vari criteri.

Nota 8.1.2: Altre classificazioni

Esistono ulteriori categorizzazioni:

- **Numero di giocatori:** Giochi a due o più giocatori (multiagente).
- **Politica di turno:** Diacronica (turni definiti) o sincronica.
- **Ambiente:** Discreto o continuo, statico o dinamico, episodico o sequenziale.
- **Somma:** Giochi a somma zero, dove il guadagno di un giocatore corrisponde alla perdita di un altro.

Esempio 8.1.1: L'agire umano

L'essere umano agisce in un ambiente che può essere descritto come continuo, dinamico, sequenziale, con scelte sincroniche e informazione imperfetta.

8.2 Algoritmi e applicazioni

Un gioco multi-agente può essere analizzato come un problema di ricerca. Tuttavia, sorgono una serie di complessità aggiuntive.

Esempio 8.2.1: Scacchi come problema di ricerca

- X : l'insieme di tutti gli stati della scacchiera.
- X_0 : lo stato iniziale.
- $SCS(x)$: le mosse legali in uno stato x .
- $T(x)$: la condizione di scacco matto.
- g : numero di mosse.

I problemi riscontrabili sono:

- L'insieme di mosse $SCS(x)$ è controllato solo in parte da un giocatore e le mosse sono spesso irreversibili.
- La terminazione non è garantita solo da $T(x)$, è necessaria una **funzione di utilità** che assegni un valore agli esiti (es. vittoria = +1, patta = 0, sconfitta = -1).
- g non è determinante.
- L'obiettivo dell'agente è di definire una strategia che raggiunga $T(x) = +1$.

Per riuscire ad inserire un gioco ad informazione perfetta in uno schema classico di ricerca è necessario considerare che:

- Esiste un avversario che va simulato.
- L'avversario cerca di minimizzare la nostra utilità.

L'albero di ricerca viene quindi sviluppato su due giocatori:

- Nodi **MAX**: rappresentano le mosse del giocatore.
- Nodi **MIN**: rappresentano le mosse dell'avversario.

L'obiettivo è raggiungere uno stato terminale di quest'albero con la massimizzazione dell'utilità. Se l'avversario inizia per primo lui sarà il nodo MAX, e noi saremo il nodo MIN; di conseguenza dovremo minimizzare la sua utilità.

8.2.1 Algoritmo Minimax

Per giochi a informazione perfetta è possibile trovare una strategia perfetta sfruttando una ricerca esaustiva dell'albero di gioco. L'algoritmo Minimax permette di trovare la strategia ottimale. L'idea sottostante è di modellare i due giocatori come un agente **MAX**, che cerca di massimizzare la propria utilità, e un agente **MIN**, che cerca di minimizzarla.

Definizione 8.2.1: Funzionamento dell'algoritmo Minimax

1. Costruzione dell'albero di gioco fino ai nodi terminali.
2. Applicazione della funzione di utilità $U(x)$ ai nodi terminali.
3. Calcolo dell'utilità dei nodi superiori propagando i valori verso la radice:
 - Se il turno è di MAX, il valore del nodo è $U(nodo_{sup}) = \max(U(\text{inf}))$.
 - Se il turno è di MIN, il valore del nodo è $U(nodo_{sup}) = \min(U(\text{inf}))$.

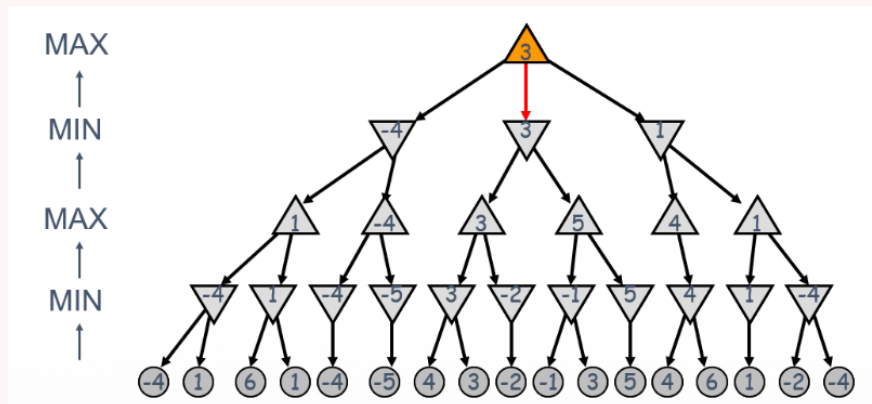


Figura 8.2: Albero di gioco per l'algoritmo Minimax.

Definizione 8.2.2: Pseudocodice Minimax

I passi descritti nell'idea dell'algoritmo (Definizione 8.2.1) possono essere formalizzati nel seguente pseudocodice:

Algoritmo 1 Pseudocodice algoritmo Minimax

```

1: function MINIMAX(nodo, agente)
2:   figli[] = SCS(nodo, agente)
3:   for all figli do                                     ▷ Itera sui nodi figli
4:     if END_test(figlio) == true then
5:       figlio.utilità = UTILITY_test(figlio)
6:     else
7:       figlio.utilità = MINIMAX(figlio, !agente)
8:     end if

9:     if agente == MAX ∧ figlio.utilità > best then
10:      best = figlio.utilità
11:    end if
12:    if agente == MIN ∧ figlio.utilità < best then
13:      best = figlio.utilità
14:    end if
15:  end for
16:  return best
17: end function

```

Proprietà e limiti

L'algoritmo Minimax presenta le proprietà seguenti:

- **Completo** per grafi finiti.
- **Ottimale** se l'avversario (MIN) gioca in modo ottimale.
- **Complessità spaziale** $O(bm)$, dove b è il fattore di diramazione e m la profondità massima dell'albero di ricerca.

Nota 8.2.1: Complessità spaziale

La complessità spaziale $O(bm)$ deriva dal fatto che viene effettuata una ricerca in profondità.

Nota 8.2.2: Complessità temporale

La complessità temporale $O(b^m)$ deriva dal fatto che in un albero di ricerca con fattore di diramazione b e profondità m , il numero totale di nodi è approssimativamente b^m .

Algoritmo Minimax ed il gioco degli scacchi

Tuttavia, ha una complessità temporale $O(b^m)$ e spaziale $O(bm)$ (con b fattore di diramazione e m profondità massima), rendendolo impraticabile per giochi complessi come gli scacchi, dove si stima $b \approx 35$ e $m \approx 100$.

Miglioramenti all'algoritmo Minimax

Per rendere l'algoritmo maggiormente utilizzabile, è necessario:

- Limitare la ricerca ad una profondità massima $d \leq m$.
- Usare una funzione di valutazione euristica per stimare l'utilità degli stati non terminali (i nodi terminali non sono più raggiunti).
- Far risalire i valori calcolati fino alla radice per determinare la mossa migliore.

Nota 8.2.3: Funzioni euristiche

Le funzioni di valutazione euristica stimano l'utilità di uno stato non terminale basandosi su caratteristiche rilevanti del gioco. Le funzioni euristiche possono essere categorizzate in:

- **Funzioni lineari pesate:** combinano diverse caratteristiche del gioco con pesi assegnati. Ad esempio $w_1f_1 + w_2f_2 + \dots + w_nf_n$.
- **Funzioni basate su pattern:** riconoscono configurazioni specifiche della scacchiera che sono vantaggiose o svantaggiose.
- **Funzioni non lineari:** ottenute tramite tecniche di "learning", ma è necessario definire un target.

Definizione 8.2.3: Quiescenza

Relativamente alle euristiche è importante parlare di **quiescenza**, ovvero la proprietà di uno stato la cui euristica di utilità non varia molto con l'applicazione operatori (legami tra fatti-condizioni e fatti-effetti, non tra stati).

L'algoritmo minimax con profondità limitata e taglio di stati quiescenti presenta:

- **Taglio di profondità:** la ricerca viene limitata a una profondità massima $d < m$. Per valutare i nodi foglia non terminali si utilizza una **funzione di valutazione euristica**.
- **Ricerca di quiescenza:** Per evitare il "problema dell'orizzonte" (un taglio che nasconde una mossa disastrosa imminente), per i nodi non "quiescenti" (stati instabili, es. durante una cattura) la ricerca viene approfondita fino a raggiungere uno stato più stabile.

Esempio 8.2.2: Problemi nel gioco degli scacchi

Un esempio di applicazione di minimax con profondità limitata e il taglio agli stati quiescenti è il gioco degli scacchi.

Se si volesse arrivare a profondità 6 in una partita di scacchi (3 mosse MAX e 3 mosse MIN), si avrebbe un numero di nodi pari a circa $35^6 \approx 1.8 \cdot 10^9$. Con un calcolatore abbastanza veloce in grado di eseguire 10^6 mosse al secondo ci vorrebbero circa 30 minuti. In aggiunta valutando fino a profondità 6 si otterrebbe un giocatore mediocre.

8.2.2 Potatura alfa-beta

La potatura (pruning) alfa-beta (McCarthy '56) è un'ottimizzazione fondamentale dell'algoritmo Minimax. Permette di "potare" interi sottoalberi che non influenzeranno la decisione finale, senza alterare il risultato.

Nota 8.2.4: Perché è possibile ottenere MAX senza osservare in modo esaustivo l'albero?

È possibile ottenere la mossa MAX senza osservare in modo esaustivo l'albero perché, durante la valutazione, si possono identificare situazioni in cui un certo ramo non può influenzare la decisione finale.

1. Dato $U(n_0) = \alpha \rightarrow$ utilità minimax del nodo n_0 su cui sceglie MAX.
2. Affinché la scelta conclusiva di MAX sia $\neq \alpha$ almeno 1 nodo n (fratello di n_0) deve avere $U(n) > \alpha$.
3. Affinché $U(n) > \alpha$ per ogni nodo n' successore di n deve valere $h(n') > \alpha$.
4. Quindi, appena un successore di n possiede $U(n') \leq \alpha$ il sottoalbero restante può essere potato.

L'algoritmo mantiene due variabili:

- **Alfa** (α): Il miglior valore trovato finora per MAX lungo il percorso dalla radice. Inizializzato a $-\infty$.
- **Beta** (β): Il miglior valore trovato finora for MIN lungo il percorso. Inizializzato a $+\infty$.

La potatura avviene quando:

- In un nodo MIN, si trova un valore $v \leq \alpha$. Il giocatore MAX ha già un'alternativa migliore, quindi eviterà questo percorso.
- In un nodo MAX, si trova un valore $v \geq \beta$. Il giocatore MIN ha già un'alternativa migliore altrove, quindi non sceglierà mai questo percorso.

8.2.3 Pseudocodice potatura alfa-beta

Algoritmo 2 Pseudocodice potatura alfa-beta

```

1: function MAX-VALUE(nodo,  $\alpha$ ,  $\beta$ )
2:   if CUTOFF-TEST(nodo) then
3:     return EVAL(nodo)
4:   end if
5:    $v \leftarrow -\infty$ 
6:   for all figlio in SCS(nodo) do
7:      $v \leftarrow \max(v, \text{MIN-VALUE}(\text{figlio}, \alpha, \beta))$ 
8:     if  $v \geq \beta$  then
9:       return  $v$ 
10:    end if
11:     $\alpha \leftarrow \max(v, \alpha)$ 
12:  end for
13:  return  $v$ 
14: end function

15: function MIN-VALUE(nodo,  $\alpha$ ,  $\beta$ )
16:   if CUTOFF-TEST(nodo) then
17:     return EVAL(nodo)
18:   end if
19:    $v \leftarrow +\infty$ 
20:   for all figlio in SCS(nodo) do
21:      $v \leftarrow \min(v, \text{MAX-VALUE}(\text{figlio}, \alpha, \beta))$ 
22:     if  $v \leq \alpha$  then
23:       return  $v$ 
24:     end if
25:      $\beta \leftarrow \min(v, \beta)$ 
26:   end for
27:   return  $v$ 
28: end function

```

Nel caso generale se n_0 è migliore di n' allora n' non verrà mai raggiunto durante il gioco e quindi tutto il sottoalbero corrispondente può essere potato.

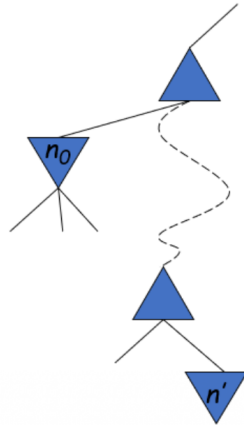


Figura 8.3: Esempio di potatura alfa-beta.

Nota 8.2.5: Efficacia della potatura

L'efficacia dipende dall'ordine di valutazione dei nodi. Con un ordinamento ottimale, la complessità temporale si riduce a $O(b^{m/2})$, raddoppiando di fatto la profondità di ricerca esplorabile. Nel caso medio è circa $O(b^{3m/4})$. In caso di ordinamento pessimo, la complessità rimane $O(b^m)$.

8.2.4 Altre problematiche da affrontare

Oltre alle problematiche relative alla profondità di ricerca e alla potatura è possibile trovare:

- Problema dell'orizzonte
- Eccessiva fiducia nell'euristica
- Eventi stocastici
- Giochi multiplayer
- Branching Factor e potenza di calcolo

8.2.5 Problema dell'orizzonte

Un lungo periodo di quiescenza (Definizione 8.2.3) può precedere un rapido e inevitabile peggioramento dell'utilità. Se il taglio in profondità è avvenuta in una specifica “zona” dell'albero, la valutazione potrebbe non riflettere questo imminente cambiamento e valutare lo stato come favorevole, portando a uno stato disastroso.

8.2.6 Eccessiva fiducia nell'euristica

Una valutazione molto irregolare tra nodi “fratelli” è rischiosa, in particolare se viene utilizzato il taglio alpha-beta. Infatti sarebbe necessario effettuare un'ulteriore ricerca nel sottoramo per poter accertare la bontà della valutazione

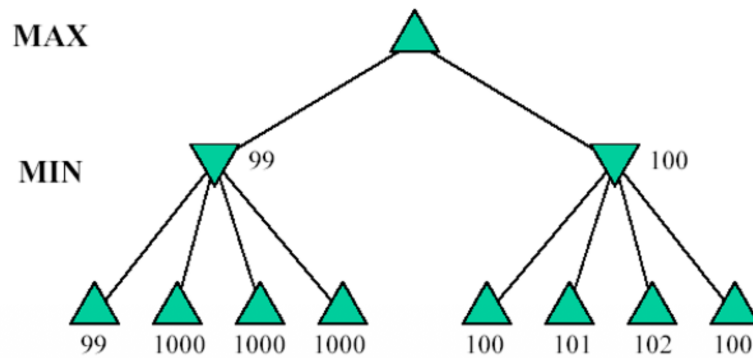


Figura 8.4: Classificazione dei giochi secondo vari criteri.

8.2.7 Eventi stocastici e ExpectiMAX

Per giochi con elementi stocastici (ovvero quando non si è in grado di sapere quale sarà l'esito di una mossa), è possibile applicare la ricerca ExpectiMAX. Invece dei nodi MIN, si introducono dei **nodi caso** (chance nodes), che calcolano il valore atteso delle possibili uscite (figli).

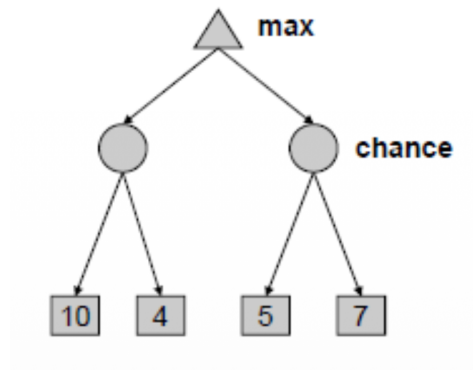


Figura 8.5: Esempio di albero ExpectiMAX.

Definizione 8.2.4: Valore atteso

Il **valore atteso** di una funzione è il suo output medio ponderato da una data distribuzione di probabilità sugli input.

Definizione 8.2.5: Utilità

Le **utilità** sono funzioni dai risultati (stati del mondo) a numeri reali che descrivono le preferenze di un agente. Le utilità potrebbero essere:

- $+1/-1$ per vittoria/sconfitta.
- GLi obbiettivi di un agente.
- Qualsiasi insieme di preferenze tra i risultati.

Definizione 8.2.6: Principio di massima utilità attesa

Un agente dovrebbe scegliere l'azione che massimizza la sua utilità attesa, data la conoscenza corrente.

8.2.8 Ricerca ExpectiMAX

Nella ricerca ExpectiMAX, si presenta un modello probabilistico di come la volontà dell'avversario, o dell'ambiente, può comportarsi in un qualsiasi stato. In questa situazione:

- Un modello che potrebbe avere una distribuzione di probabilità (es. uniforme) sulle mosse legali.
- Il modello potrebbe essere più sofisticato e richiedere una grande quantità di calcoli.
- Esiste un nodo per risultato al di fuori dal controllo dell'agente.
- Il modello potrebbe portare a probabili azioni contraddittorie.

Definizione 8.2.7: Pseudocodice ExpectiMAX**Algoritmo 3** Pseudocodice algoritmo ExpectiMAX

```

1: function VALUE( $s$ )
2:   if  $s$  is a max node then
3:     return MAXVALUE( $s$ )
4:   end if
5:   if  $s$  is an exp node then
6:     return EXPVALUE( $s$ )
7:   end if
8:   if  $s$  is a terminal node then
9:     return evaluation( $s$ )
10:  end if
11: end function

12: function MAXVALUE( $s$ )
13:   $values = [VALUE(s') \text{ for } s' \text{ in successors}(s)]$ 
14:  return max( $values$ )
15: end function

16: function EXPVALUE( $s$ )
17:   $values = [VALUE(s') \text{ for } s' \text{ in successors}(s)]$ 
18:   $weights = [probability(s, s') \text{ for } s' \text{ in successors}(s)]$ 
19:  return expectation( $values, weights$ )
20: end function

```

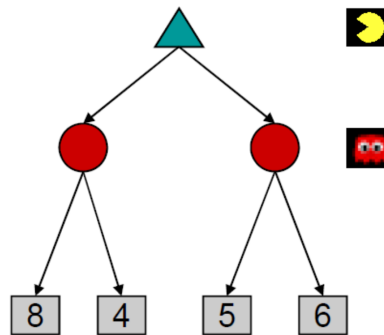


Figura 8.6: Esempio di albero ExpectiMAX.

Giochi multiplayer e algoritmo MAX^N

Per giochi con più di due giocatori, dove non è presente accordo tra questi, è possibile generalizzare Minimax con l'algoritmo MAX^N . Per funzionare è necessario introdurre delle assunzioni:

- I giocatori muovono a turni
- Ogni giocatore mira a massimizzare il proprio utile
- Ogni giocatore 'è indifferente all'utile degli avversari

La funzione di valutazione ora restituisce un vettore di utilità $U(s) = \langle u_1, u_2, \dots, u_n \rangle$, dove $u_i = U(p_i, s)$ è l'utilità per il giocatore p_i allo stato del gioco s . L'algoritmo in questione è di tipo depth-first search come Minimax e fa risalire la n -upla che massimizza $U(p_n)$ quando è il turno del giocatore p_n .

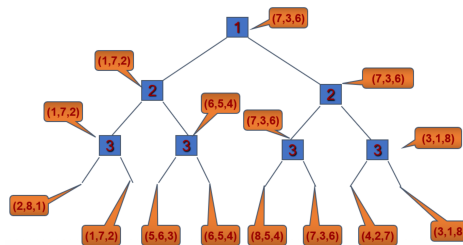


Figura 8.7: Esempio di albero MAX^N per tre giocatori.

8.2.9 Algoritmo Paranoid

Un'alternativa più semplice a MAX^N è l'algoritmo **Paranoid**, che assume che tutti gli altri giocatori formino una coalizione contro MAX, minimizzando la sua utilità. Questo riduce il problema a un gioco a due giocatori, permettendo di usare la potatura alfa-beta, ma non garantisce l'ottimalità.

Nota 8.2.6: La vera sfida

La vera sfida nei giochi multiplayer non è tanto la ricerca della mossa ottimale, ma la modellazione degli avversari e delle loro strategie. Infatti un uomo non utilizza semplicemente la ricerca come metodo principale ma:

- Prima parte dai GOAL (non ben definiti).
- A ritroso costruisce SOTTOGOAL
- pianifica: azioni $7 \rightarrow$ sottogoal $7 \rightarrow$ goal
- Usa la ricerca per raggiungere obiettivi locali
- Ha capacità "istintive" di escludere le scelte inutili: riduce enormemente il branching factor

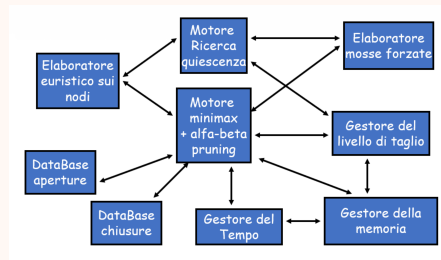


Figura 8.8: Esempio di giocatore di scacchi artificiale.

8.3 Apprendimento per rinforzo

8.3.1 Introduzione

L'apprendimento per rinforzo (reinforcement learning) è una tecnica di apprendimento automatico che mira a creare **agenti autonomi** capaci di scegliere azioni in un **ambiente** per raggiungere un determinato **obiettivo**. L'ambiente fornisce un feedback sotto forma di **ricompense**, e l'agente cerca di massimizzare una nozione di **premio cumulativo**.

L'agente non viene istruito su cosa fare, ma impara attraverso l'interazione, ricevendo ricompense (positive o negative) in base alle sue azioni.

Esistono due paradigmi principali:

- **Model-free:** L'agente impara una funzione di valore o una politica direttamente dall'esperienza, senza costruire un modello esplicito dell'ambiente.
- **Model-based:** L'agente impara un modello dell'ambiente (funzione di transizione e di ricompensa) e lo usa per pianificare le azioni.

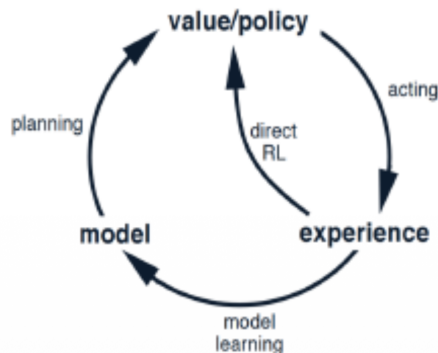


Figura 8.9: Relazione tra learning, planning e acting.

8.3.2 Model learning

Nel model-based, il modello è tipicamente un Processo Decisionale di Markov (MDP), definito dalla tupla $\langle S, A, R, T, \gamma \rangle$ parametrizzato secondo ν . In questa situazione è necessario assumere che è presente un'indipendenza condizionale tra le transizioni e le ricompense: $P[s_{t+1}, r_{t+1} | s_t, a_t] = P[s_{t+1} | s_t, a_t] P[r_{t+1} | s_t, a_t]$

L'apprendimento del modello è di tipo supervisionato e consiste nell'apprendere:

- La **funzione di ricompensa** $R(s, a)$ tramite regressione.
- La **distribuzione di transizione** $P(s' | s, a)$ come problema di stima della densità.

Nota 8.3.1: Scelta della famiglia di modelli

Per l'apprendimento del modello, è necessario scegliere una famiglia appropriata di modelli parametrizzati. Tra le opzioni disponibili vi sono: modelli di ricerca tabellare, aspettativa lineare, gaussiana lineare, processo gaussiano, rete di credenze profonde, ecc. Una volta selezionata la famiglia di modelli, si sceglie una funzione di perdita (loss) appropriata, come l'errore quadratico medio o la divergenza KL, per ottimizzare i parametri del modello in modo da minimizzare tale perdita.

8.3.3 Ricerca basata su simulazione

Dato un modello (esatto o appreso), i metodi di ricerca basata su simulazione identificano l'**azione migliore** attraverso simulazioni in avanti, costruendo un **albero di ricerca** a partire dallo stato corrente. Questo approccio è efficiente perché risolve un MDP subottimale solo per lo stato attuale.

Una volta acquisita l'esperienza simulata, è possibile utilizzare un approccio model-free, come i metodi Monte-Carlo (MC). Nello specifico, dato un modello M e una politica di simulazione π , in un algoritmo di ricerca MC, per ogni azione $a \in A$ vengono simulati K episodi seguendo la politica π . Il valore $Q(s_t, a)$ viene calcolato come la media delle ricompense ottenute da tali simulazioni. Infine, viene scelta l'azione che massimizza $Q(s_t, a)$.

Monte-Carlo Tree Search (MCTS)

MCTS è una famiglia di algoritmi di ricerca basata su simulazione, particolarmente efficace in giochi con spazi di ricerca molto grandi. Non richiede conoscenza del dominio, ma solo la capacità di campionare gli esiti delle azioni (black-box).

Questa famiglia di algoritmi si basa su due principi fondamentali:

1. Il vero valore di uno stato può essere stimato utilizzando la media dei valori ottenuti da simulazioni casuali.
2. Tali valori possono essere utilizzati per aggiustare iterativamente le politiche di scelta secondo una strategia best-first, permettendo di concentrarsi sulle regioni dello spazio di ricerca con rendimento e valore migliori.

In ogni nodo dell'albero vengono memorizzate le seguenti statistiche:

- Conteggio totale delle visite $N(s)$.
- Conteggio specifico per ogni coppia stato-azione $N(s, a)$.
- Valori stimati tramite Monte-Carlo $Q(s, a)$.

L'albero di ricerca parziale è costruito **progressivamente** utilizzando il nodo corrente come radice di un nuovo sottalbero. Ogni iterazione dell'algoritmo MCTS consiste in quattro fasi principali:

1. **Selezione:** A partire dalla radice, si seleziona ricorsivamente un nodo foglia promettente usando una tree policy.
2. **Espansione:** Se il nodo foglia non è terminale, si espande l'albero aggiungendo uno o più nodi figli.

3. **Simulazione:** Da un nuovo nodo, si esegue una simulazione cercando di produrre una stima degli esiti (outcomes).
4. **Retropropagazione:** L'esito della simulazione viene usato per aggiornare le statistiche (es. numero di vittorie e visite) di tutti i nodi attraversati nella fase di selezione.

Le varianti dell'algoritmo modificano due politiche principali per migliorare l'utilizzo della memoria, per ottenere una potatura migliore o per avere delle statistiche più complesse. Le politiche sono:

- **Tree policy:** per scegliere le azioni da poter applicare in ogni nodo e quindi le loro statistiche
- **Roll out policy:** per decidere come eseguire le simulazioni dei nodi foglia.

Algoritmo 4 General MCTS algorithm

```

1: function MCTS( $s_0$ )
2:   Create root node  $v_0$  corresponding to  $s_0$ 
3:   while within computational budget do
4:      $v_k \leftarrow \text{TreePolicy}(v_0)$ 
5:      $\Delta \leftarrow \text{Simulation}(v_k)$ 
6:      $\text{Backprop}(v_k, \Delta)$ 
7:   end while
8:   return  $\arg \max_a Q(s_0, a)$ 
9: end function

```

Algoritmo 5 Greedy Tree policy

```

1: function TREEPOLICY( $v$ )
2:    $v_{next} \leftarrow v$ 
3:   if  $|\text{Children}(v_{next})| \neq 0$  then
4:      $a \leftarrow \arg \max_{a \in A} Q(v, a)$ 
5:      $v_{next} \leftarrow \text{nextState}(v, a)$ 
6:      $v_{next} \leftarrow \text{TreePolicy}(v_{next})$ 
7:   end if
8:   return  $v_{next}$ 
9: end function

```

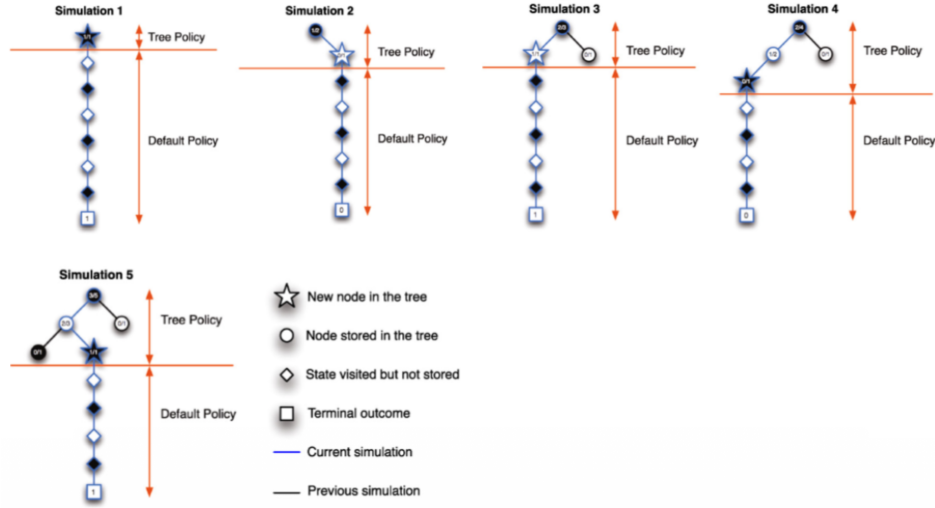


Figura 8.10: Esempio di simulazione MCTS.

Nota 8.3.2: Upper Confidence Bound (UCB)

La politica greedy può portare a delle scelte molto **subottimali** che portano l'algoritmo a evitare **azioni** con un solo o pochi non significativi **risultati negativi**. Questo fenomeno è dovuto essenzialmente alla **incertezza** circa il valore/probabilità reale di tali azioni negative. Per risolvere tale problema si può applicare il principio di **ottimalità sotto incertezza** al MCTS usando UCB (limite superiore di confidenza). Invece di selezionare l'azione in modo greedy, si seleziona l'azione che massimizza il **limite superiore di confidenza** il cui valore è dato da:

$$Q(s, a) + \sqrt{\frac{2 \ln N(s)}{N(s, a)}}$$

dove $Q(s, a)$ è il valore stimato dell'azione, $N(s)$ è il numero di visite al nodo padre, e $N(s, a)$ è il numero di volte che l'azione è stata scelta.

Algoritmo 6 Upper Confidence Tree policy

```

1: function TREEPOLICY( $v$ )
2:    $v_{next} \leftarrow v$ 
3:   if  $|\text{Children}(v_{next})| \neq 0$  then
4:      $a \leftarrow \arg \max_{a \in A} \left( Q(v, a) + \sqrt{\frac{2 \ln N(v)}{N(v, a)}} \right)$ 
5:      $v_{next} \leftarrow \text{nextState}(v, a)$ 
6:      $v_{next} \leftarrow \text{TreePolicy}(v_{next})$ 
7:   end if
8:   return  $v_{next}$ 
9: end function

```

Go e AlphaGo

Il Go è un gioco da tavolo a informazione perfetta, deterministico, con un fattore di diramazione enorme ($b > 200$), che lo ha reso per decenni una grande sfida per l'IA.

AlphaGo, sviluppato da DeepMind, ha sconfitto i migliori giocatori umani combinando MCTS con reti neurali profonde:

- Una **Policy Network** per guidare la selezione delle mosse, riducendo l'ampiezza della ricerca.
- Una **Value Network** per valutare le posizioni, riducendo la profondità della ricerca.

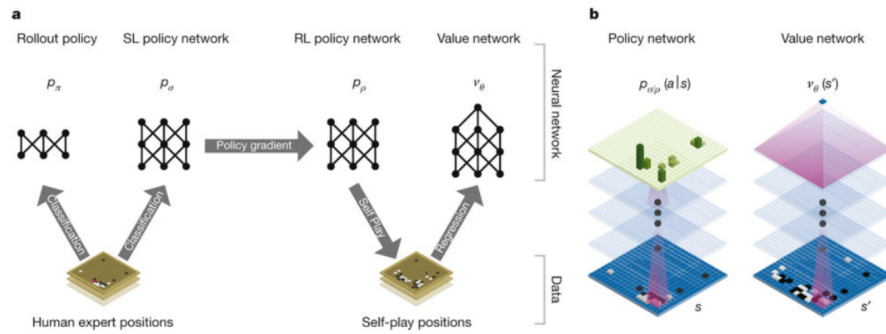


Figura 8.11: Esempio di simulazione MCTS.

Nota 8.3.3: AlphaGO Zero

Le versioni successive, come **AlphaGO Zero**, hanno ulteriormente migliorato l'approccio imparando esclusivamente tramite auto-gioco (self-play), senza dati umani.